COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science

TechAware AI-Based Feedback System

A Thesis
for the Degree of

Bachelor of Science in Computer Science

Submitted By

Ali Ahmad (FA22-BCS-163)
Nimra Farooq (FA22-BCS-160)

Supervisor

Sir Rizwan Ali

Session: Spring 2022-2026

Dedication

I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey.

I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity.

Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

Certificate of Approval

This is to certify that the Final Year Project titled **"TechAware AI-Based Feedback System"** has been developed by **Ali Ahmad (FA22-BCS-163)** and **Nimra Farooq (FA22-BCS-160)** under the supervision of **Sir Rizwan Ali** and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science.

_____

Supervisor

_____

External Examiner

_____

Head of Department
Department of Computer Science

Plagiarism and AI Content Certificate

Date: _____

It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the **Final Year Project Report** submitted by:

- **Ali Ahmad (FA22-BCS-163)**
- **Nimra Farooq (FA22-BCS-160)**

titled:

"TechAware AI-Based Feedback System"

is [**XX%**] which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as [**XX%**]. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

Supervisor Name: Sir Rizwan Ali _____

Designation: _____

Signature: _____

Department: Department of Computer Science

University: COMSATS University Islamabad, Vehari Campus

Executive Summary

Traditional student feedback collection methods in educational institutions rely heavily on manual paper-based forms or generic online surveys. These approaches suffer from low response rates, delayed processing, lack of analytical depth, and an inability to provide timely, actionable insights to educators. As a result, teachers often receive feedback that is either too late to act upon or too unstructured to derive meaningful conclusions.

The TechAware AI-Based Feedback System addresses these challenges by providing a comprehensive, web-based platform that automates the entire feedback lifecycle — from collection to analysis to actionable reporting. The system integrates Natural Language Processing (NLP) for real-time sentiment analysis, enabling automatic classification of student feedback as Positive (Understood), Negative (Not Understood), or Neutral. Based on the analysis, AI-powered suggestions are generated to help teachers improve their teaching methodologies.

The system supports four distinct user roles: Students, Teachers, Administrators, and Batch Advisors. Students submit structured feedback after lectures, which includes star ratings across multiple dimensions (clarity, punctuality, material quality, communication, and overall satisfaction) along with free-text comments. Teachers access a dedicated dashboard to view aggregated analytics, sentiment distributions, and AI-generated improvement suggestions. Administrators manage the entire academic infrastructure, including departments, batches, sections, courses, timetables, and user accounts. Batch Advisors receive automated alerts when teacher performance falls below defined thresholds.

The system is built using Python Flask as the backend framework, SQLAlchemy ORM with SQLite for data persistence, TextBlob for sentiment analysis, and HTML/CSS/JavaScript for the frontend. Additional integrations include Chart.js for data visualization, ReportLab for PDF report generation, and SMTP for automated email notifications.

The expected impact of this system is to enhance teaching quality through data-driven feedback, reduce administrative overhead in feedback management, and promote a culture of continuous improvement in academic institutions.

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
CDN	Content Delivery Network
CRUD	Create, Read, Update, Delete
CSRF	Cross-Site Request Forgery
CSS	Cascading Style Sheets
DB	Database
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
NLP	Natural Language Processing
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
SDD	Software Design Document
SMTP	Simple Mail Transfer Protocol
SRS	Software Requirements Specification
SQL	Structured Query Language
TLS	Transport Layer Security
UAT	User Acceptance Testing
UI	User Interface
UX	User Experience

Table of Contents

List of Tables

List of Figures

Chapter 1
Introduction

1.1 Introduction

In modern educational institutions, the quality of teaching and learning is closely linked to the effectiveness of feedback mechanisms between students and educators. Feedback allows educators to identify areas of improvement in their teaching methodologies and helps institutions maintain academic standards. However, traditional approaches to collecting student feedback — such as paper-based forms, verbal discussions, or end-of-semester surveys — are often inefficient, time-consuming, and fail to provide real-time, actionable insights.

Current industry practices include generic online survey platforms such as Google Forms, SurveyMonkey, and institutional Learning Management Systems (LMS) with built-in feedback modules. While these tools offer basic data collection capabilities, they lack intelligent analysis features such as sentiment analysis, automated suggestion generation, and integration with academic management workflows. The feedback collected through these platforms typically requires manual review and interpretation, which delays corrective actions and limits the impact of student input on teaching quality.

Key challenges in existing feedback systems include low response rates due to lack of engagement, delayed feedback loops that prevent timely instructional adjustments, absence of NLP-based analysis to extract meaningful trends from free-text responses, and limited integration with institutional administrative systems such as timetable and attendance management.

The TechAware AI-Based Feedback System addresses these challenges by providing a comprehensive web-based platform that automates the entire feedback lifecycle. The system collects structured and unstructured feedback from students after each lecture, performs real-time sentiment analysis using Natural Language Processing (NLP) through the TextBlob library, generates AI-powered teaching improvement suggestions, and presents actionable analytics through role-based dashboards. The platform integrates with academic management functions including department, batch, section, course, timetable, and attendance management, ensuring that only students who were present in a lecture can submit feedback for that session.

The system supports four distinct user roles — Student, Teacher, Administrator, and Batch Advisor — each with tailored interfaces and functionality. This modular, role-based design ensures secure access, streamlined workflows, and comprehensive oversight of the feedback process.

1.2 Vision Statement

The TechAware AI-Based Feedback System envisions creating a data-driven academic feedback ecosystem where educators receive continuous, intelligent, and actionable insights to improve their teaching effectiveness. The system targets students, teachers, department administrators, and batch advisors within university environments, providing each stakeholder with role-specific tools and analytics.

The core problem the system addresses is the disconnect between student experiences in lectures and the timely availability of structured, analyzed feedback to educators. By combining automated feedback collection with NLP-based sentiment analysis and AI-generated teaching suggestions, the system transforms raw student opinions into measurable performance indicators and concrete improvement recommendations.

The long-term vision is to establish TechAware as a standard feedback management platform for educational institutions, promoting a culture of continuous improvement in teaching quality. The system innovates by integrating attendance-verified feedback submission, time-based feedback windows triggered by lecture completion, aggregated sentiment analysis, and automated performance monitoring with threshold-based alerts to batch advisors.

1.3 Related System Analysis

Several existing systems address aspects of student feedback collection and analysis. This section critically evaluates three widely used platforms and compares them with the proposed TechAware system.

Google Forms is a general-purpose survey tool commonly used by educators for feedback collection. It provides basic form creation, response collection, and spreadsheet-based data export. However, Google Forms lacks built-in sentiment analysis, does not support role-based access for different stakeholders, provides no AI-generated suggestions, and has no integration with academic management systems such as timetable or attendance.

Moodle Feedback Module is the feedback component of the Moodle LMS, offering course-level survey and feedback collection within the LMS ecosystem. While it supports some analytics and is integrated with course management, it does not provide NLP-based sentiment analysis, lacks automated email notification systems for teachers, does not generate AI-powered improvement suggestions, and does not support attendance-based feedback verification.

SurveyMonkey is a commercial survey platform with advanced question types, templates, and basic analytics. It supports various distribution methods and response analysis. However, it is a generic tool not designed for academic feedback, lacks educational workflow integration, does not provide sentiment analysis on textual feedback, and requires paid plans for advanced features.

Table 1.1: Comparative Analysis of Existing Systems

Application Name	**Identified Weaknesses**	**Proposed Improvements**
Google Forms	• No sentiment analysis • No role-based access • No academic integration	• Automated NLP-based sentiment analysis • Role-based dashboards for 4 user types • Integrated timetable and attendance management
Moodle Feedback	• No AI-powered suggestions • No automated email notifications • No attendance-verified feedback	• AI-generated teaching improvement suggestions • SMTP-based automated email system • Only present students can submit feedback
SurveyMonkey	• Not designed for academic use • No free-text analysis • Paid plans required	• Purpose-built for educational institutions • TextBlob-based feedback text analysis • Open-source, no licensing cost

1.4 Project Deliverables

The TechAware AI-Based Feedback System produces the following tangible outputs, each serving a specific purpose within the project scope. The deliverables encompass the complete software product along with all associated documentation and testing artifacts.

The web application is the primary deliverable, consisting of a fully functional Flask-based system with role-based portals for students, teachers, administrators, and batch advisors. The application includes feedback collection forms, sentiment analysis engine, analytics dashboards, attendance management, timetable management, and automated email notification capabilities.

The documentation deliverables ensure compliance with academic requirements and support future maintenance. The source code is structured following modular design principles with clear separation of models, routes, templates, and static assets.

List of Deliverables:

1. TechAware Web Application – A complete Flask-based web application with four role-based portals, feedback collection, sentiment analysis, analytics dashboards, and administrative management features.

2. Software Requirements Specification (SRS) – A comprehensive document detailing all func-

tional and non-functional requirements, user classes, use cases, and external interface specifications.

3. Software Design Document (SDD) – A detailed design document including architectural design, UML models, data flow diagrams, database schema, and data dictionary.
4. Test Cases and Testing Report – A structured set of unit, integration, system, and user acceptance test cases with documented results.
5. User Documentation – Setup instructions, configuration guide, and README file for system deployment and usage.
6. Source Code Repository – Complete, organized project source code uploaded to a version control platform with proper documentation.

1.5 System Limitations / Constraints

The TechAware system operates within certain technical, operational, and resource constraints that define its current scope. These limitations are acknowledged and documented to guide future development and set appropriate user expectations.

- The sentiment analysis engine relies on TextBlob, which provides moderate accuracy for English text. Feedback written in Urdu, Roman Urdu, or mixed languages may not be accurately classified.
- The current deployment uses SQLite database, which is suitable for development and small-scale deployment but may require migration to PostgreSQL or MySQL for large-scale institutional use.
- The email notification system depends on Gmail SMTP service and requires a configured app password. Institutional SMTP servers may need additional configuration.
- The system is designed as a web application and does not include a native mobile application. Mobile access is supported through responsive web design.
- Real-time notifications are implemented through polling (auto-refresh every 5 seconds) rather than WebSocket-based push notifications, which may introduce minor delays.
- The AI suggestion engine uses keyword-based analysis rather than advanced machine learning models, limiting the depth of generated recommendations.

1.6 Tools and Technologies

The selection of tools and technologies for the TechAware system was guided by project requirements, development team expertise, and suitability for web-based feedback management with NLP capabilities. Each technology was evaluated for reliability, community support, and compatibility with the overall system architecture.

Python was selected as the primary programming language due to its extensive library ecosystem for NLP and web development. Flask was chosen as the web framework for its lightweight, modular design that supports rapid prototyping while maintaining flexibility for custom implementations. SQLAlchemy ORM provides database abstraction, enabling seamless interaction with SQLite during development with the option to migrate to PostgreSQL for production deployment. TextBlob was selected for sentiment analysis due to its simplicity and built-in lexicon-based approach, which eliminates the need for custom model training while providing adequate accuracy for educational feedback classification.

Table 1.2: Tools and Technologies Used in the Project

Tool / Technology	**Version**	**Purpose**
Python	3.8+	Primary programming language for backend logic and NLP processing
Flask	2.3.3	Lightweight web framework for routing, templating, and API development
Flask-SQLAlchemy	3.0.5	ORM for database abstraction and model-based data access
SQLite	3.x	Relational database for development and small-scale deployment
TextBlob	0.17.1	NLP library for sentiment analysis (polarity and subjectivity scoring)
ReportLab	4.0.4	PDF generation library for creating formatted feedback reports
bcrypt	4.0.1	Cryptographic password hashing for secure credential storage
Werkzeug	2.3.7	WSGI utility library providing password hashing and HTTP utilities
Chart.js	4.x (CDN)	JavaScript charting library for dashboard data visualization
HTML5 / CSS3 / JavaScript	Latest	Frontend technologies for responsive user interface development
Gmail SMTP	–	Email delivery service for automated feedback and alert notifications
Visual Studio Code	1.80+	Integrated development environment for code editing and debugging
Git	2.x	Version control system for source code management

1.7 Relevance to Course Modules

This section demonstrates how the academic knowledge acquired through the Computer Science degree program was applied in the practical implementation of the TechAware system.

1.7.1 Programming Fundamentals

The TechAware system extensively utilizes core programming concepts including object-oriented programming through Python class definitions for database models, modular design with separate files for models, routes, and database initialization, control structures for sentiment classification logic, and function-based code organization with decorators for authentication and role-based access control. These fundamental programming principles ensured clean, maintainable code architecture.

1.7.2 Data Structures and Algorithms

Data structures and algorithms are applied in multiple system components. List operations are used for managing feedback collections and email queues. Dictionary data structures store session information and API response formatting. The sentiment analysis algorithm processes text through sequential keyword matching against predefined category lists. Sorting algorithms are applied for reverse-chronological feedback display, and searching operations are used for database queries through SQLAlchemy's query interface.

1.7.3 Database Management Systems

Database management principles are central to the TechAware system. The database design follows Third Normal Form (3NF) to minimize redundancy across 12 interrelated tables. Entity-Relationship modeling guided the definition of primary keys, foreign keys, and unique constraints. Join operations through SQLAlchemy relationships enable complex queries across User, Feedback, Course, and Teacher tables. Transaction management ensures data consistency during concurrent feedback submissions.

1.7.4 Software Engineering

Software engineering methodologies were applied throughout the project lifecycle. The Agile iterative development model guided phase-wise implementation. Requirements were formally documented using IEEE-style functional and non-functional requirement specifications. UML diagrams (Class, Sequence, Activity, State Transition) were created following UML 2.0 standards. A structured testing strategy covering unit, integration, system, and acceptance testing validated system correctness. The SRS and SDD documents follow standard software engineering documentation practices.

1.7.5 Other Relevant Courses

Concepts from Artificial Intelligence were applied through TextBlob-based Natural Language Processing for sentiment analysis and keyword-driven AI suggestion generation. Web Development principles guided the creation of responsive HTML/CSS interfaces with AJAX-based asynchronous data loading. Computer Networks knowledge was applied in SMTP email integration and HTTP/HTTPS communication protocols. Human-Computer Interaction principles influenced the design of intuitive, role-based dashboards with consistent visual elements and real-time feedback mechanisms.

1.8 Chapter Summary

This chapter introduced the background and context of the TechAware AI-Based Feedback System, highlighting existing challenges in student feedback collection and the need for an automated, intelligent solution. The vision and strategic objectives of the system were outlined to establish its intended impact on teaching quality improvement. A comparative analysis of related systems (Google Forms, Moodle, SurveyMonkey) identified key limitations in current solutions that the proposed system addresses. Project deliverables, system constraints, selected technologies, and academic relevance were discussed to provide a comprehensive introduction to the project. This chapter establishes the foundation for the detailed problem definition presented in the next chapter.

Chapter 2
Problem Definition

2.1 Problem Statement

Educational institutions currently lack efficient, automated mechanisms for collecting and analyzing student feedback on teaching quality. Traditional feedback methods such as paper-based forms, end-of-semester surveys, and generic online questionnaires suffer from low response rates, delayed processing, and an inability to provide real-time analytical insights. Teachers receive feedback too late to make timely instructional adjustments, and the unstructured nature of text-based responses makes manual review impractical. Administrators lack a centralized platform to monitor teaching performance across departments and courses. The absence of sentiment analysis and AI-powered suggestion generation means that valuable patterns and actionable trends within student feedback remain undetected, resulting in missed opportunities for continuous improvement in teaching quality.

2.2 Proposed Solution Overview

The TechAware AI-Based Feedback System is a web-based platform that automates the complete student feedback lifecycle — from collection to analysis to reporting. The system collects structured ratings and free-text feedback from students after each lecture session, performs real-time sentiment analysis using TextBlob NLP, classifies feedback as Positive (Understood), Negative (Not Understood), or Neutral, and generates AI-powered teaching improvement suggestions based on keyword analysis. The system integrates academic management features including department, batch, section, course, timetable, and attendance management to ensure attendance-verified feedback submission. Four distinct user portals — Student, Teacher, Administrator, and Batch Advisor — provide role-specific dashboards with relevant analytics, automated email notifications, and PDF report generation capabilities.

2.3 Objectives of the Proposed System

The following measurable objectives define the intended outcomes of the TechAware system. Each objective aligns directly with challenges identified in the Problem Statement.

Business Objectives (BO):

- BO-1: Develop a web-based feedback collection system that enables students to submit structured and textual feedback after each lecture session.
- BO-2: Implement real-time NLP-based sentiment analysis to automatically classify student feedback into Positive, Negative, or Neutral categories.

- BO-3: Provide AI-powered teaching improvement suggestions to educators based on feedback content analysis.
- BO-4: Ensure role-based access control with dedicated portals for Students, Teachers, Administrators, and Batch Advisors.
- BO-5: Automate the feedback notification workflow through SMTP-based email alerts and dashboard-based performance monitoring.

2.4 Scope of the System

The TechAware system is a web-based application accessible through modern web browsers on desktop and mobile devices. The system targets university-level educational institutions and supports four user roles: Students, Teachers, Administrators, and Batch Advisors. Core functionalities include feedback collection with multi-dimensional star ratings, NLP-based sentiment analysis, AI suggestion generation, analytics dashboards with Chart.js visualizations, PDF report export, SMTP email notifications, and comprehensive academic management (departments, batches, sections, courses, timetables, attendance). The system does not include a native mobile application, video-based feedback, parent portal access, or multi-language NLP support. Deployment is scoped for single-institution use with SQLite database.

2.5 System Architecture and Modules

The TechAware system follows a three-tier layered architecture consisting of a Presentation Layer (HTML/CSS/JavaScript frontend templates), an Application Layer (Python Flask backend with business logic and NLP processing), and a Data Layer (SQLAlchemy ORM with SQLite database). The system is designed using modular principles with clearly defined module boundaries, ensuring loose coupling between components and enabling independent development, testing, and maintenance of each module.

2.5.1 Module 1: User Management and Authentication

This module handles user registration, authentication, authorization, and profile management across all four user roles. Administrators can create user accounts with assigned roles. Teachers and students authenticate using email and password credentials. Password security is implemented using bcrypt hashing through the Werkzeug library. Session management ensures that authenticated users can access only role-appropriate system features.

Functional Requirements:

- FE-1: The system shall allow administrators to create user accounts with assigned roles (Student, Teacher, Admin, Batch Advisor).
- FE-2: The system shall authenticate users using email and hashed password credentials before granting access to role-specific portals.

2.5.2 Module 2: Academic Management

This module manages the organizational hierarchy of the institution including departments, batches, sections, courses, and teacher-course assignments. Administrators can perform full CRUD operations on all academic entities. The module supports batch-subject assignment and teacher-section-course mapping to establish the relationships needed for timetable and feedback management.

Functional Requirements:

- FE-1: The system shall enable administrators to create, read, update, and delete departments, batches, sections, and courses.
- FE-2: The system shall support assignment of subjects to batches and teachers to specific course-section combinations.
- FE-3: The system shall maintain timetable records linking courses, teachers, sections, and time slots with conflict detection.

2.5.3 Module 3: Feedback Collection and Sentiment Analysis

This module implements the core functionality of the system. Students submit feedback through a structured form including star ratings for clarity, punctuality, material quality, communication, and overall satisfaction, along with free-text comments. The TextBlob NLP library analyzes the textual feedback to calculate polarity and subjectivity scores, classifying each response as Understood (positive), Not Understood (negative), or Neutral. AI-powered suggestions are generated using keyword-based analysis of feedback content.

Functional Requirements:

- FE-1: The system shall collect student feedback through a structured form with multi-dimensional ratings and free-text input.
- FE-2: The system shall perform real-time sentiment analysis using TextBlob and classify feedback based on polarity thresholds.
- FE-3: The system shall generate AI-powered teaching improvement suggestions based on keyword analysis of feedback text.

2.5.4 Module 4: Administration and Analytics Dashboard

This module provides comprehensive administrative control and analytics capabilities. The admin dashboard displays system-wide statistics, teacher performance comparisons, feedback sentiment distributions, and custom report generation. Teachers access a dedicated dashboard showing their feedback analytics, AI suggestions, and performance metrics. Batch advisors receive automated alerts when teacher performance drops below defined thresholds.

Functional Requirements:

- FE-1: The system shall provide administrators with a dashboard displaying system-wide statistics and teacher performance analytics.

- FE-2: The system shall generate visual analytics using Chart.js including doughnut charts for sentiment distribution.

2.5.5 Module 5: Email Notification and Report Generation

This module handles automated communication through the SMTP email system and PDF report generation through ReportLab. After each feedback submission, the system sends an email notification to the relevant teacher containing feedback details, sentiment analysis results, and AI suggestions. The system also supports PDF export of feedback reports with formatted tables and statistics. Batch advisors receive automated email alerts for low-performing teachers.

Functional Requirements:

- FE-1: The system shall send automated email notifications to teachers after each feedback submission using SMTP.
- FE-2: The system shall generate downloadable PDF reports containing feedback statistics and analysis summaries.

2.6 Assumptions and Dependencies

The following assumptions and dependencies were identified during system design:

- The system assumes users have stable internet connectivity for accessing the web application and receiving email notifications.
- The system depends on Gmail SMTP service availability for email delivery; institutional SMTP servers may be substituted.
- The system requires Python 3.8 or higher with Flask 2.3.3 and associated libraries installed on the deployment server.
- Students are assumed to possess basic web browsing skills to access the feedback form and submit responses.
- Teachers are assumed to have active email accounts configured in the system for receiving notifications.
- The TextBlob library requires the NLTK corpora to be downloaded for sentiment analysis functionality.

2.7 Chapter Summary

This chapter defined the core problem addressed by the TechAware AI-Based Feedback System — the lack of efficient, automated, and analytically intelligent student feedback mechanisms in educational institutions. The proposed solution was outlined as a web-based platform integrating

NLP-based sentiment analysis, AI-powered suggestions, and comprehensive academic management. Clear business objectives were established to ensure alignment with identified challenges. The system scope, three-tier architectural overview, and five major modules were presented to define structural boundaries and functional responsibilities. Key assumptions and dependencies were also documented to clarify operational constraints.

This chapter provides the problem foundation and sets the direction for the detailed requirement analysis presented in the next chapter.

Chapter 3
Requirement Analysis

3.1 Introduction to Requirement Analysis

This chapter presents a structured and formal specification of system requirements for the TechAware AI-Based Feedback System. The purpose of this chapter is to clearly define what the system shall accomplish before proceeding to the system design phase. The requirements documented in this chapter serve as a contractual foundation for implementation, validation, and system testing.

The requirement analysis includes requirement elicitation techniques, identification of stakeholders, use case modeling, functional requirements, non-functional requirements, and external interface requirements. All requirements are written in measurable and testable format using standard requirement engineering principles.

3.2 Requirement Elicitation Techniques

This section describes the techniques used to gather and refine system requirements. Requirement elicitation ensures alignment between stakeholder expectations and system capabilities.

Requirements for this project were identified using the following techniques:

- Stakeholder interviews with faculty members and students to capture feedback process expectations and pain points.
- Literature review of existing feedback management systems and sentiment analysis research.
- Analysis of existing solutions (Google Forms, Moodle, SurveyMonkey) to identify functional gaps.
- Examination of NLP libraries (TextBlob, NLTK) for sentiment analysis feasibility.
- Observation of current feedback workflow processes at COMSATS University Islamabad, Vehari Campus.

3.3 User Classes and Characteristics

This section identifies different user categories interacting with the system. Each user class is defined along with their responsibilities, access privileges, and technical proficiency level. This classification supports role-based access control and requirement mapping.

Table 3.1: User Classes and Characteristics

User Class	**Description**	**Technical Level**
Administrator	Manages system configuration, user accounts, departments, batches, sections, courses, timetables, and views all feedback analytics.	Advanced
Teacher	Reviews student feedback, views sentiment analysis results and AI suggestions, marks student attendance, manages own profile, and accesses performance dashboard.	Intermediate
Student	Submits feedback for attended lectures through a structured form with ratings and text comments. Views personal timetable and feedback history.	Basic
Batch Advisor	Monitors teacher performance across assigned batches, receives automated alerts for low-performing teachers, and views batch-level statistics.	Intermediate

3.4 System Overview

The TechAware system operates within a three-tier layered architecture designed to support modular interaction between user interfaces, business logic, and data storage. The presentation layer consists of HTML/CSS/JavaScript templates rendered by the Flask templating engine. The application layer implements all business logic including feedback processing, sentiment analysis, suggestion generation, and email notification in Python Flask. The data layer uses SQLAlchemy ORM with SQLite database to manage 12 interrelated tables.

The system supports role-based interactions through four distinct portals and ensures secure communication between internal modules and external services including Gmail SMTP for email delivery and Chart.js CDN for data visualization.

3.5 Use Case Model

The use case model represents high-level system interactions between actors and system functionalities. It defines system boundaries and captures functional behavior from a user perspective.

3.5.1 Use Case Diagram

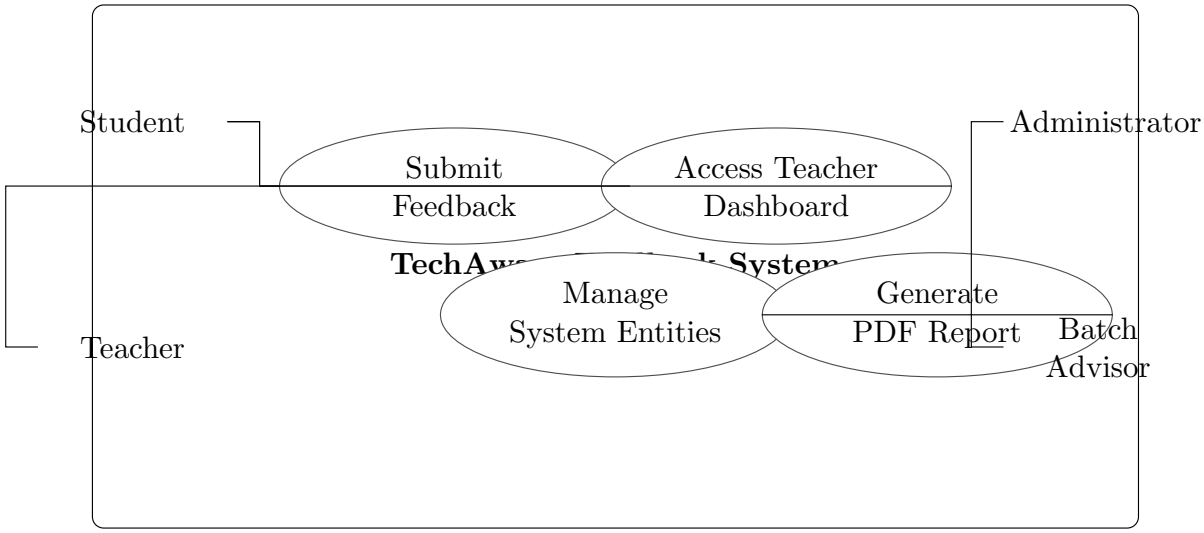


Figure 3.1: Use Case Diagram of TechAware AI-Based Feedback System

The above diagram illustrates the interaction between actors (Student, Teacher, Administrator, Batch Advisor) and major system functionalities including feedback submission, sentiment analysis, dashboard access, attendance management, and report generation.

3.5.2 Use Case Descriptions

This section provides detailed tabular specifications of major use cases. These use cases form the basis for deriving functional requirements.

3.5.2.1 Module 1: Student Feedback Submission

Below are the detailed use cases for the student feedback module.

Table 3.2: UC-1.1: Submit Feedback

Use Case ID	UC-1.1
Use Case Name	Submit Feedback
Actors	Student
Description	This use case describes how a student submits feedback for an attended lecture session.
Trigger	Student accesses the feedback form after lecture completion.
Preconditions	Student is logged in and was marked present in the lecture session.
Postconditions	Feedback is stored with sentiment analysis results, and email notification is sent to the teacher.
Normal Flow	1. Student selects the pending feedback session. 2. Student fills in ratings (clarity, punctuality, material quality, communication, overall). 3. Student enters text feedback (minimum 10 characters). 4. System validates all required fields. 5. System performs sentiment analysis using TextBlob. 6. System generates AI-powered suggestions. 7. System stores feedback and sends email notification to teacher. 8. Confirmation message with sentiment result is displayed.
Alternative Flow	If validation fails (missing fields or feedback text too short), the system displays appropriate error messages and the form is not submitted.

Table 3.3: UC-1.2: Teacher Dashboard Access

Use Case ID	UC-1.2
Use Case Name	Access Teacher Dashboard
Actors	Teacher
Description	This use case describes how a teacher accesses the analytics dashboard to view feedback results.
Trigger	Teacher navigates to the dashboard page after login.
Preconditions	Teacher is authenticated with valid credentials.
Postconditions	Dashboard displays statistics, charts, AI suggestions, and feedback list.
Normal Flow	1. Teacher enters email and password on login page. 2. System validates credentials against database. 3. System redirects to teacher dashboard. 4. Dashboard loads statistics (understood, not understood, neutral counts). 5. Chart.js renders doughnut chart for sentiment distribution. 6. AI suggestions panel displays generated recommendations. 7. Feedback list shows individual entries in reverse chronological order.
Alternative Flow	If credentials are invalid, the system displays an error message and the teacher remains on the login page.

Table 3.4: UC-1.3: Admin System Management

Use Case ID	UC-1.3
Use Case Name	Manage System Entities
Actors	Administrator
Description	This use case describes how an administrator manages departments, batches, sections, courses, teachers, and students.
Trigger	Administrator accesses the admin management panel.
Preconditions	Administrator is authenticated with admin role credentials.
Postconditions	System entities are created, updated, or deleted as requested.
Normal Flow	1. Administrator logs in to admin portal. 2. Administrator selects entity type (Department, Batch, Section, etc.). 3. Administrator performs CRUD operation (Create, Read, Update, Delete). 4. System validates input and updates database. 5. Confirmation message is displayed.
Alternative Flow	If validation fails (duplicate entries, missing required fields), the system displays error messages.

3.5.3 Event-Response Table

The event-response table defines how the system reacts to significant internal and external events.

Table 3.5: Event-Response Table

Event	**Source**	**System Response**
User Login Attempt	Student / Teacher / Admin	Validate credentials against database and grant or deny access to role-specific portal.
Feedback Submission	Student	Perform sentiment analysis, generate AI suggestions, store feedback, and send email notification to teacher.
Lecture Completion	Teacher	Open feedback window for present students and send feedback request notifications.
Attendance Marking	Teacher	Record student attendance status (Present/Absent/Late) for the lecture session.
PDF Export Request	Teacher / Admin	Generate PDF report with feedback statistics using ReportLab and return downloadable file.
Low Performance Alert	System	Send automated email notification to Batch Advisor when teacher performance drops below threshold.

3.6 Functional Requirements

Functional requirements define the core services and behaviors that the system shall provide. These requirements are derived directly from the use case model presented in Section 3.5. Each requirement is uniquely identified, measurable, and written using formal specification language.

All functional requirements follow IEEE-style structure and are categorized by system modules.

3.6.1 Module 1: User Management

3.6.1.1 FR-1.1 User Authentication

Table 3.6: Functional Requirement – User Authentication

Identifier	FR-1.1
Title	User Authentication
Requirement	The system shall authenticate users (Students, Teachers, Administrators, Batch Advisors) using email and hashed password credentials before granting access to role-specific portals.
Source	Stakeholder Requirement
Rationale	Ensures secure, role-based access to system functionalities.
Dependencies	Database Connectivity, bcrypt Password Hashing
Priority	High

3.6.2 Module 2: Feedback Collection and Sentiment Analysis

3.6.2.1 FR-2.1 Feedback Submission

Table 3.7: Functional Requirement – Feedback Submission and Sentiment Analysis

Identifier	FR-2.1
Title	Submit Feedback with Sentiment Analysis
Requirement	The system shall accept student feedback consisting of multi-dimensional star ratings and free-text comments, perform real-time sentiment analysis using TextBlob (polarity and subjectivity scoring), classify the feedback as Understood/Not Understood/Neutral, generate keyword-based AI suggestions, and store all results in the database.
Source	System Requirement Specification
Rationale	Enables automated analysis of student feedback to provide actionable insights to teachers.
Dependencies	TextBlob Library, Database Connectivity, User Authentication Module
Priority	High

3.6.3 Module 3: Reporting and Email Notification

3.6.3.1 FR-3.1 Email Notification and Report Generation

Table 3.8: Functional Requirement – Email Notification and Reporting

Identifier	FR-3.1
Title	Automated Email Notifications and PDF Reports
Requirement	The system shall send automated email notifications to teachers after each feedback submission containing feedback details, sentiment results, and AI suggestions. The system shall also generate downloadable PDF reports with formatted feedback statistics using ReportLab.
Source	Business Requirement
Rationale	Supports timely communication of feedback results and enables offline report access.
Dependencies	Gmail SMTP Service, ReportLab Library, Feedback Module
Priority	Medium

3.7 Non-Functional Requirements

Non-functional requirements define quality attributes that describe how the system performs. These requirements are measurable and support system reliability, usability, performance, security, and maintainability.

3.7.1 Performance Requirements

- PER-1: The system shall load the main dashboard within 2 seconds under normal operating conditions.
- PER-2: The system shall support at least 100 concurrent users.
- PER-3: Sentiment analysis processing shall be completed within 0.5 seconds per feedback submission.
- PER-4: API response time shall not exceed 1 second for standard operations.

3.7.2 Security Requirements

- SEC-1: The system shall implement secure authentication using email and hashed password credentials.
- SEC-2: Passwords shall be encrypted using bcrypt hashing algorithm via Werkzeug.
- SEC-3: Sensitive data shall be transmitted using HTTPS protocol in production deployment.

- SEC-4: Role-based access control shall be enforced to restrict users to their designated portals.
- SEC-5: CSRF protection shall be implemented to prevent cross-site request forgery attacks.

3.7.3 Reliability Requirements

- REL-1: The system shall maintain at least 99% uptime during academic sessions.
- REL-2: System failures shall not result in loss of submitted feedback data.
- REL-3: Database backup mechanisms shall be implemented to support disaster recovery.

3.7.4 Usability Requirements

- USE-1: The interface shall be intuitive and navigable with minimal training for all user roles.
- USE-2: Clear and meaningful error messages shall be displayed for all validation failures.
- USE-3: The system shall follow consistent UI/UX design principles across all portals.
- USE-4: The application shall be responsive and accessible on desktop and mobile browsers.

3.7.5 Scalability Requirements

- SCA-1: The database design shall support migration from SQLite to PostgreSQL or MySQL for larger deployments.
- SCA-2: The system architecture shall allow horizontal scaling through load balancing.
- SCA-3: The modular design shall support addition of new modules without major restructuring.

3.7.6 Maintainability Requirements

- MAIN-1: The system shall follow modular architecture with separated models, routes, and templates.
- MAIN-2: Source code shall be documented with comments and a comprehensive README file.
- MAIN-3: Future updates shall be integrated without major system restructuring through the use of ORM and templating patterns.

3.8 External Interface Requirements

This section defines how the system interacts with external entities including users, hardware devices, software systems, and communication networks.

3.8.1 User Interface Requirements

- UI-1: The system shall provide a graphical web-based user interface accessible through modern browsers (Chrome, Firefox, Safari, Edge).
- UI-2: Standard font sizes of 12–14pt shall be used to ensure readability across all pages.
- UI-3: Layout shall be responsive across desktop and mobile devices using CSS media queries.
- UI-4: Navigation shall remain consistent across all modules with a unified header and menu structure.

3.8.2 Hardware Interface Requirements

- HI-1: The system shall operate on standard computing devices including desktops, laptops, tablets, and smartphones.
- HI-2: The server shall require minimum 4GB RAM and standard CPU for Flask application hosting.
- HI-3: No specialized hardware (GPU, biometric devices) is required for system operation.

3.8.3 Software Interface Requirements

- SI-1: The system shall interact with SQLite relational database through SQLAlchemy ORM.
- SI-2: TextBlob NLP library shall be integrated for sentiment analysis processing.
- SI-3: ReportLab library shall be integrated for PDF report generation.
- SI-4: Chart.js CDN shall be integrated for frontend data visualization (doughnut charts).

3.8.4 Communication Interface Requirements

- CI-1: The system shall use HTTP protocol for development and HTTPS for production deployment.
- CI-2: RESTful API endpoints shall be used for frontend-backend communication via AJAX.
- CI-3: SMTP protocol with TLS encryption shall be used for email notification delivery through Gmail (port 587).
- CI-4: JSON format shall be used for all API request and response data exchange.

3.9 Assumptions and Dependencies

This section lists assumptions made during requirement analysis and system design planning.

- The system assumes stable internet connectivity for web application access and email delivery.
- Gmail SMTP service is assumed to remain available for email notifications; app password configuration is required.
- Users are assumed to possess basic digital literacy to operate web browser interfaces.
- Python 3.8+ runtime environment with pip package manager is assumed to be available on the deployment server.
- TextBlob NLTK corpora are assumed to be downloaded and available for sentiment analysis.
- Chart.js CDN is assumed to be accessible for frontend chart rendering.

3.10 Requirement Traceability Matrix

This matrix maps project objectives to functional requirements to ensure alignment and completeness.

Table 3.9: Requirement Traceability Matrix

Objective	**Use Case**	**Functional Requirement**
BO-1	UC-1.1	FR-2.1 (Feedback Submission)
BO-2	UC-1.1	FR-2.1 (Sentiment Analysis)
BO-3	UC-1.1	FR-2.1 (AI Suggestions)
BO-4	UC-1.2, UC-1.3	FR-1.1 (User Authentication)
BO-5	UC-1.2	FR-3.1 (Email Notification and Reporting)

Chapter Summary

This chapter presented a structured requirement analysis of the TechAware AI-Based Feedback System. It defined requirement elicitation techniques used during the project, identified four user classes (Student, Teacher, Administrator, Batch Advisor), and presented the use case model with three detailed use case descriptions and an event-response table. Functional requirements were specified across three modules: User Management, Feedback Collection and Sentiment Analysis, and Reporting and Email Notification. Non-functional requirements covering performance, security, reliability, usability, scalability, and maintainability were documented. External interface requirements for user, hardware, software, and communication interfaces were specified. A traceability matrix was included to ensure alignment between business objectives and implementation requirements. These requirements form the formal foundation for the system design presented in the next chapter.

Chapter 4
Software Design

4.1 Introduction to Software Design

This chapter presents the detailed software design of the TechAware AI-Based Feedback System. It translates the requirements identified in Chapter 3 into structured architectural, behavioral, and data models. The purpose of this chapter is to provide a clear blueprint for system implementation, ensuring maintainability, scalability, and modular development.

This chapter includes architectural design, UML models, data flow models, and database design applicable to the web-based feedback management system.

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodology

This project follows a Layered Architecture design methodology combined with Object-Oriented Design (OOD) principles. The system is organized into three distinct layers:

- Presentation Layer (HTML/CSS/JavaScript templates)
- Application Layer (Flask backend with business logic and NLP processing)
- Data Layer (SQLAlchemy ORM with SQLite database)

The selected methodology ensures modularity, loose coupling, reusability, and scalability. The layered approach was chosen because the web-based nature of the system requires clear separation between user interface rendering, business logic processing (including sentiment analysis), and data persistence. Design principles followed include Separation of Concerns, DRY (Don't Repeat Yourself), and modular code organization with separate files for models, database initialization, and application routes.

4.2.2 Software Process Model

The project follows the **Agile** model with iterative development cycles. The Agile approach was selected due to the evolving nature of feedback system requirements and the need for continuous stakeholder feedback during development.

Development was organized into six iterative phases: Core Infrastructure (database and models), Academic Management (departments, batches, sections, courses), Timetable and Attendance System, Feedback Collection and Sentiment Analysis, Portal Development and Analytics, and Enhancements (email notifications, reporting, security). Each iteration included requirement refinement, implementation, and testing, ensuring that functional components were delivered incrementally and validated before proceeding to the next phase.

4.3 System Overview

This section provides a high-level overview of the TechAware system components and their interactions with external entities. The context diagram below illustrates the system boundary, identifying external actors (Student, Teacher, Administrator, Batch Advisor) and external systems (Gmail SMTP, Chart.js CDN) that interact with the application.

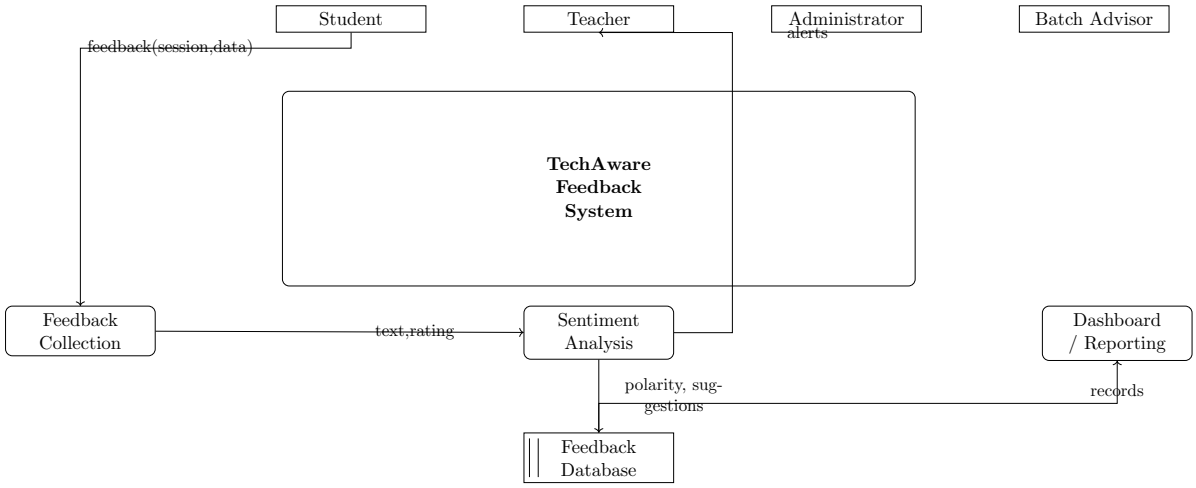


Figure 4.1: Context Diagram of TechAware AI-Based Feedback System

The context diagram shows the system as a central process with data flows to and from four actor types and two external services. Students provide feedback data; the system returns sentiment analysis results. Teachers receive feedback analytics and AI suggestions. Administrators manage all system entities. The system communicates with Gmail SMTP for email delivery and retrieves Chart.js from CDN for dashboard visualization.

4.4 Architectural Design

This section describes the high-level architecture of the TechAware system. The system follows a three-tier web application architecture.

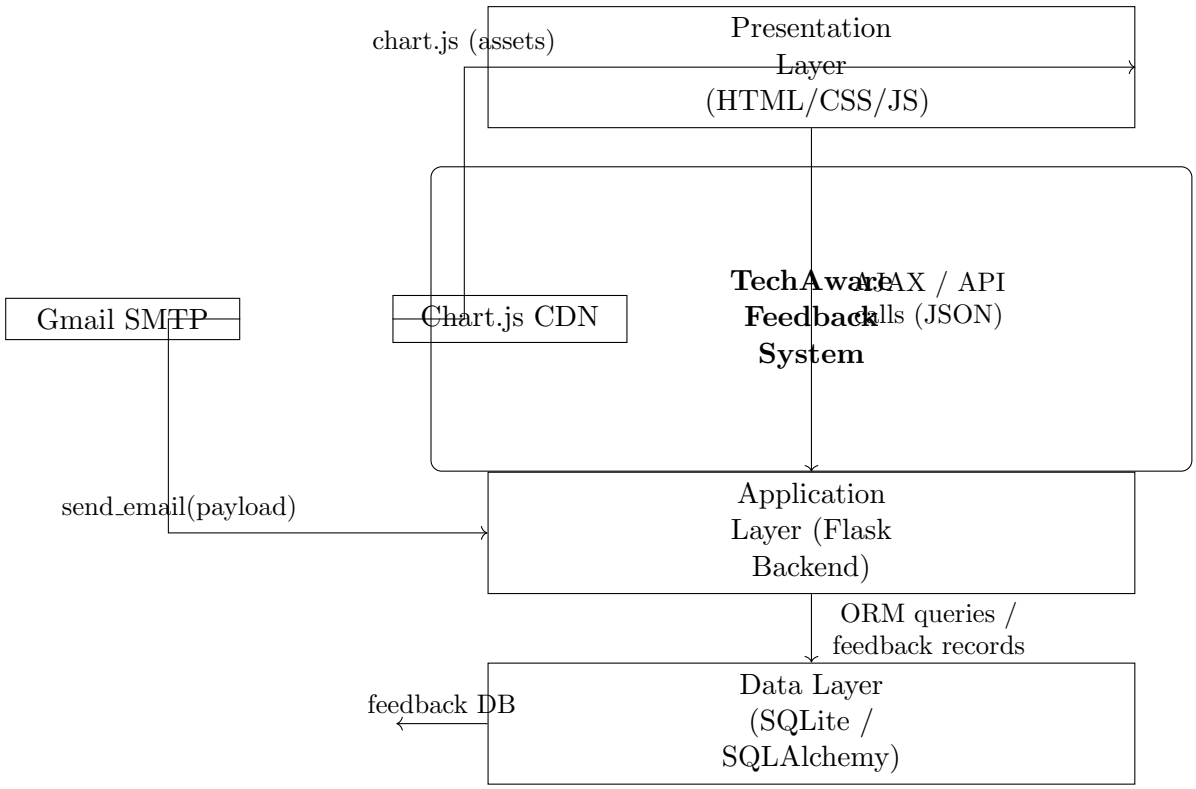


Figure 4.2: System Architecture Diagram of TechAware AI-Based Feedback System

The architecture consists of the following layers:

- **Presentation Layer**: HTML5/CSS3/JavaScript frontend templates rendered by Flask's Jinja2 templating engine. Includes responsive layouts, AJAX-based asynchronous data loading, and Chart.js for data visualization.
- **Application Layer**: Python Flask backend implementing all business logic including user authentication, feedback processing, TextBlob sentiment analysis, AI suggestion generation, SMTP email notification, and ReportLab PDF generation.
- **Data Layer**: SQLAlchemy ORM providing database abstraction over SQLite. Contains 12 interrelated models (User, Department, Batch, Section, Student, Course, Teacher, BatchSubject, CourseTeacher, Feedback, Attendance, Timetable).

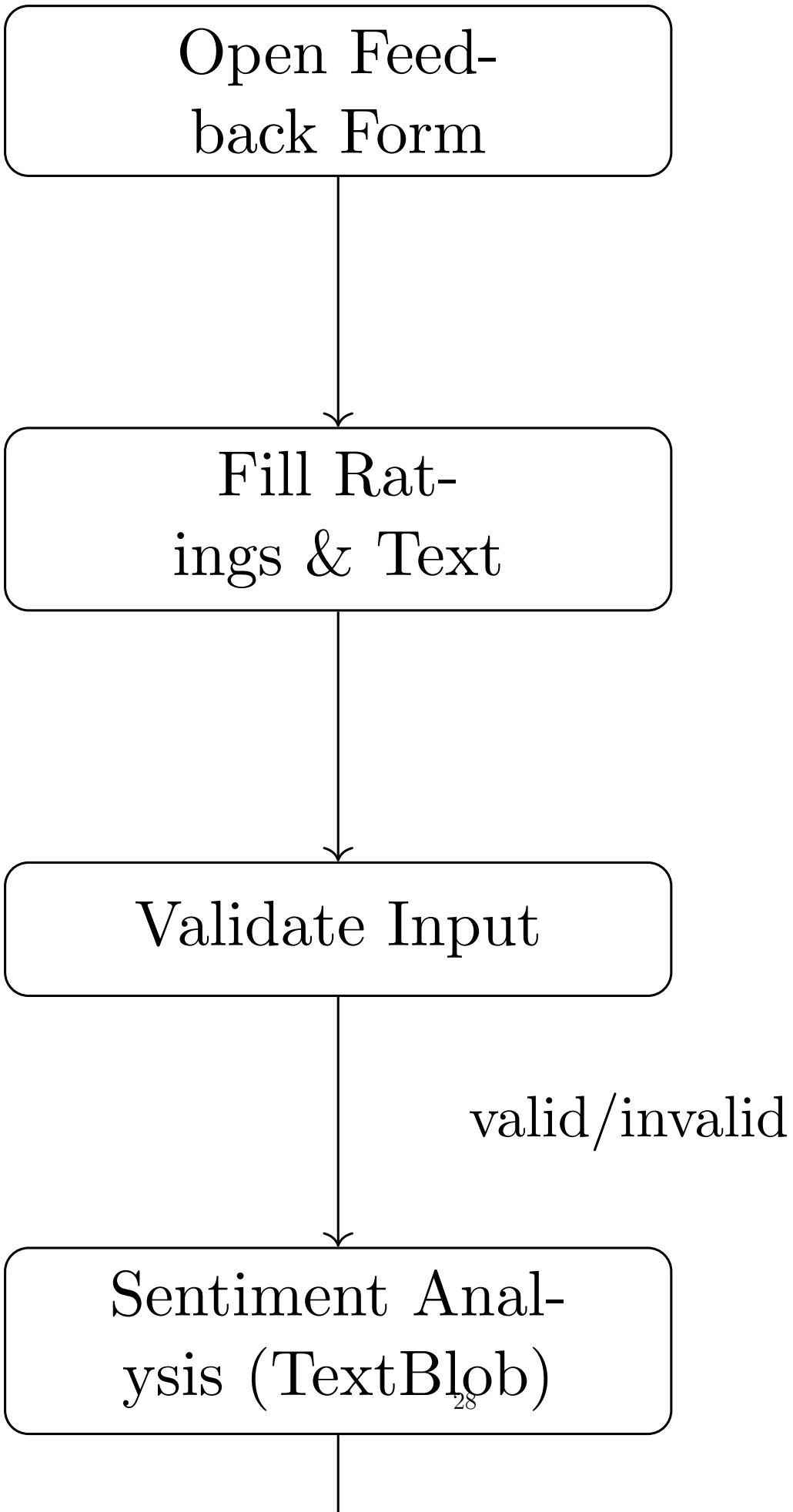
The interaction between layers follows a request-response pattern: the browser sends HTTP requests to Flask routes, which execute business logic, interact with the database through SQLAlchemy, and return rendered HTML or JSON responses.

4.5 Design Models

This section presents detailed design models of the system. These models describe system behavior, structure, and interactions using UML diagrams. The models help developers understand system workflow before implementation.

4.5.1 Activity Diagrams

Activity diagrams represent the workflow of different modules of the system. Each core module has a separate activity diagram illustrating its operational flow.


Open Feed-
back Form
Fill Rat-
ings & Text
Validate Input
valid/invalid
Sentiment Anal-
ysis (TextBlob)

4.5.1.1 Module 1: User Authentication

This diagram illustrates the user login workflow. The process begins when a user enters credentials on the login page. The system validates the email and password against the database. If authentication succeeds, the user is redirected to their role-specific portal (Student Dashboard, Teacher Dashboard, or Admin Panel). If authentication fails, an error message is displayed and the user remains on the login page.

Figure 4.4: Activity Diagram – User Authentication Module

4.5.1.2 Module 2: Feedback Collection and Analysis

This diagram illustrates the complete feedback submission and analysis workflow. The student accesses the feedback form, selects the lecture session, provides ratings and text feedback, and submits the form. The system validates inputs, performs sentiment analysis using TextBlob, generates AI suggestions based on keyword matching, stores the feedback in the database, sends an email notification to the teacher, and displays the sentiment result to the student.

Figure 4.5: Activity Diagram – Feedback Collection Module

4.5.1.3 Module 3: Admin Management Operations

This diagram illustrates administrative CRUD operations including department creation, batch management, section assignment, course creation, teacher management, and timetable configuration. The admin selects an entity type, performs the desired operation (Create/Read/Update/Delete), the system validates input, updates the database, and displays confirmation.

Figure 4.6: Activity Diagram – Admin Operations

4.5.2 Class Diagram

The class diagram shows the static structure of the TechAware system, illustrating all major classes, their attributes, methods, and relationships.

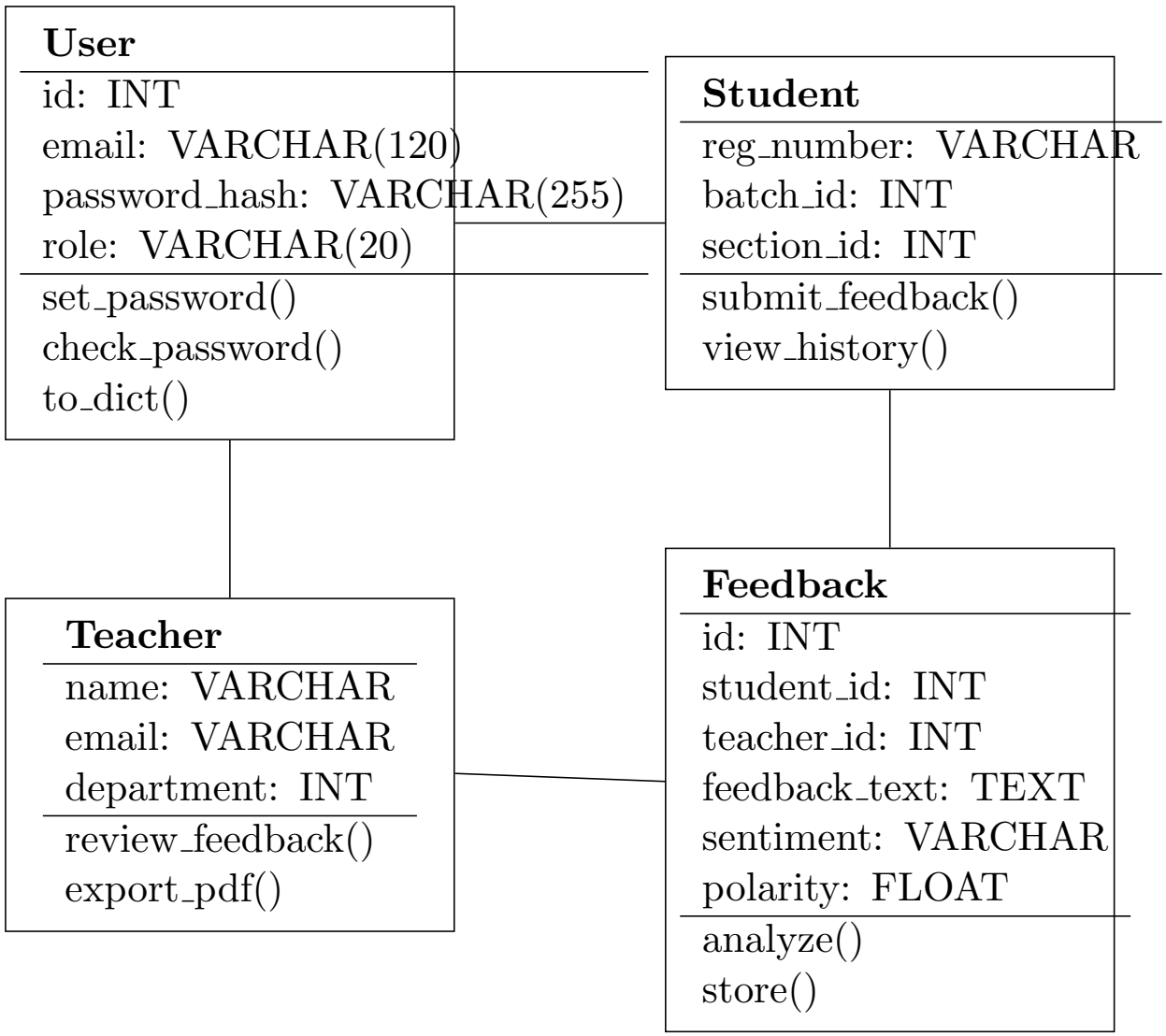


Figure 4.4: Class Diagram of TechAware AI-Based Feedback System

The class diagram includes the following key classes and their relationships:

- **User**: Handles authentication with attributes (username, email, password_hash, role) and methods (set_password, check_password, to_dict).
- **Student**: Extends user functionality with registration_number, batch, section, and department associations. Has one-to-many relationships with Feedback and Attendance.
- **Teacher**: Contains name, email, subject, department, and designation. Has one-to-many relationship with Feedback and CourseTeacher assignments.
- **Feedback**: Core entity linking Student, Course, Teacher, and LectureSession with attributes for ratings, sentiment, polarity, subjectivity, and AI suggestions.
- **Department**: Aggregation relationship with Batch, Student, Teacher, and Course entities.

4.5.3 Sequence Diagrams

Sequence diagrams show interaction between objects over time for key system operations.

4.5.3.1 Sequence Diagram – User Login

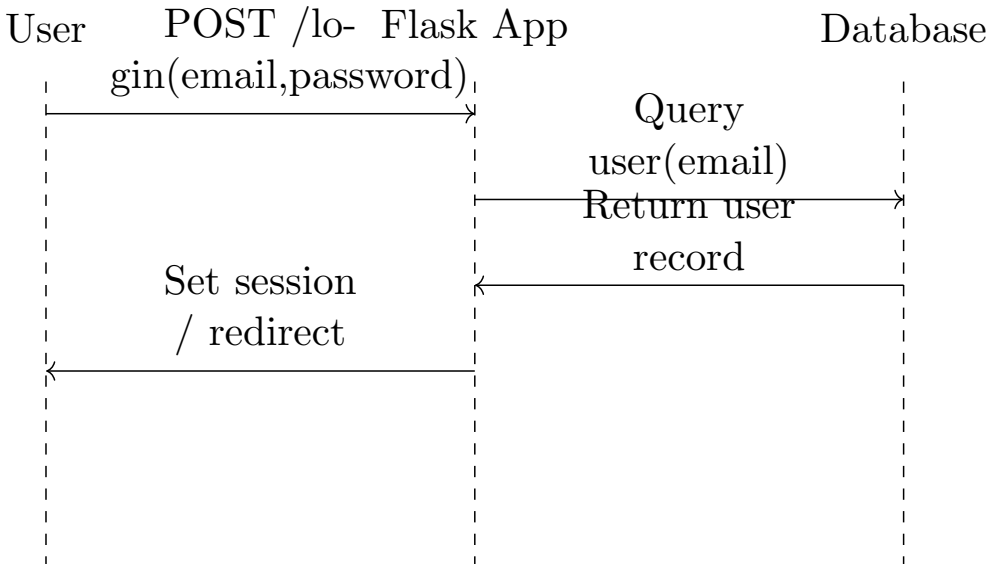


Figure 4.5: Sequence Diagram – User Login

The login sequence proceeds as follows: The user sends credentials (email, password) to the Flask application via HTTP POST request. The application queries the database through SQLAlchemy to retrieve the user record. The password hash is validated using Werkzeug's check_password_hash function. If valid, the application creates a session and redirects to the role-specific dashboard. If invalid, an error response is returned.

4.5.3.2 Sequence Diagram – Feedback Submission

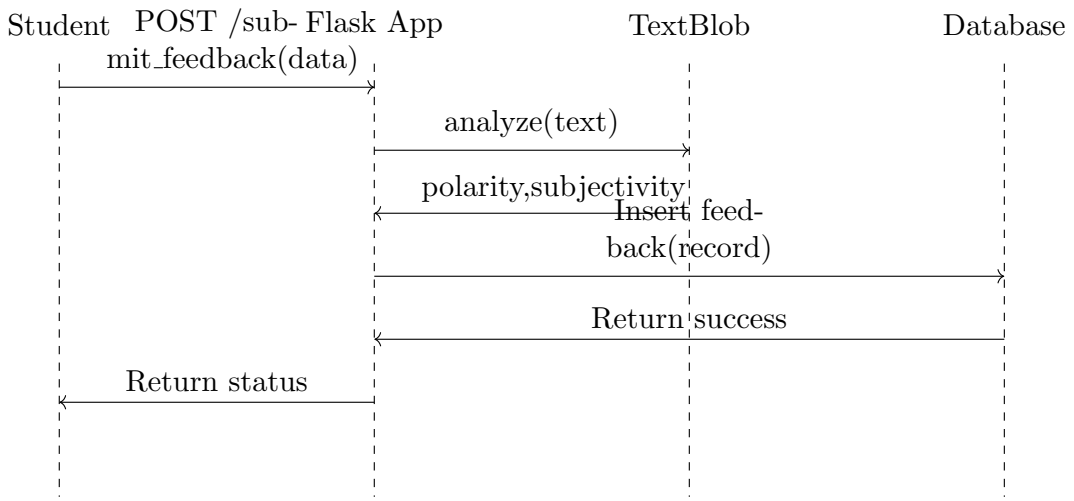


Figure 4.6: Sequence Diagram – Feedback Submission and Analysis

The feedback submission sequence involves: Student submits feedback form data via AJAX POST request. Flask application validates input fields. TextBlob library processes feedback text to calculate polarity and subjectivity scores. The system classifies sentiment based on polarity thresholds. AI suggestion engine performs keyword analysis and generates recommendations. Feedback record is stored in database via SQLAlchemy. Email notification is sent through Gmail SMTP. JSON response with sentiment result and suggestions is returned to the student's browser.

4.5.4 State Transition Diagrams

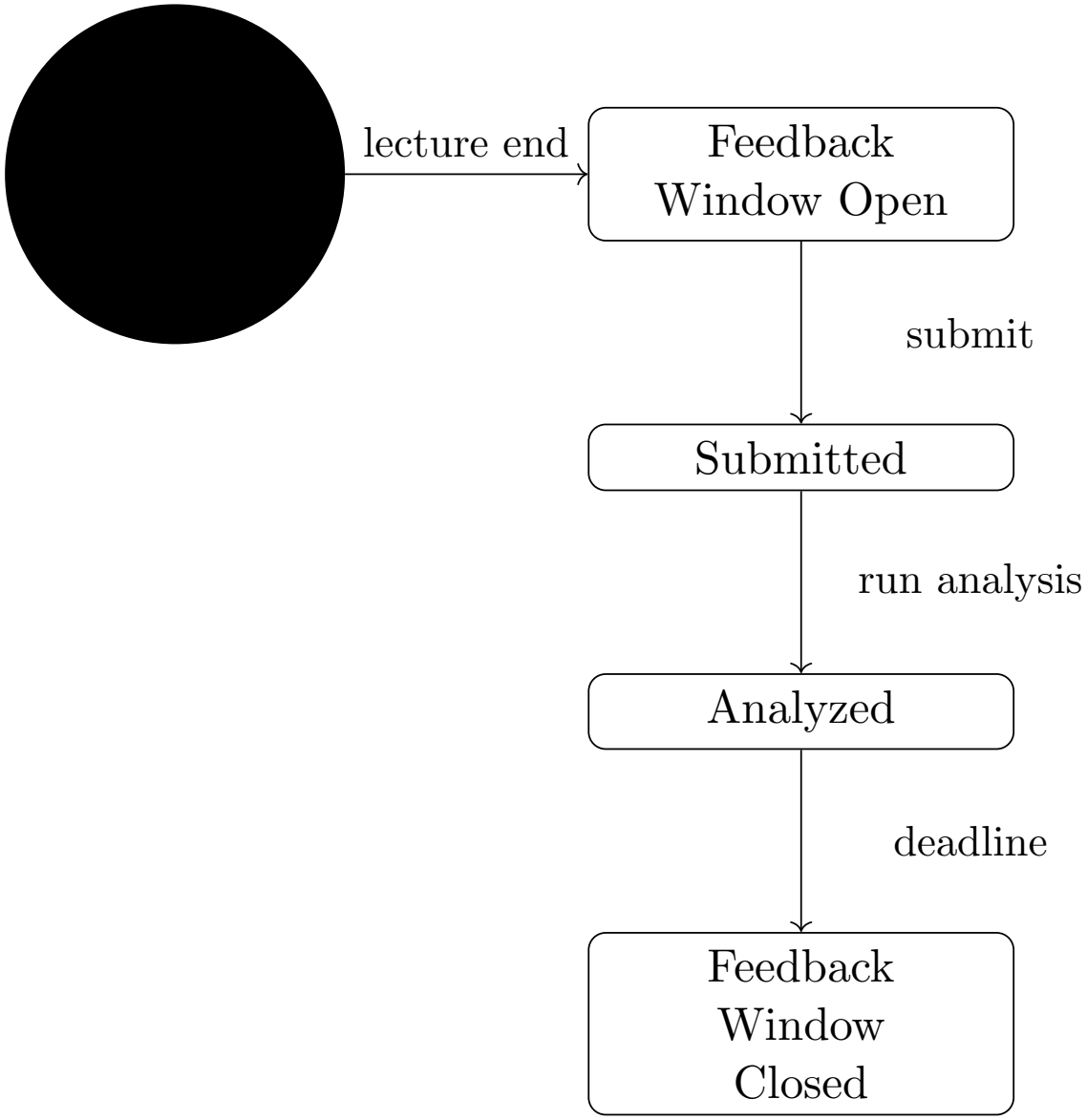


Figure 4.7: State Transition Diagram – Feedback Lifecycle

The state transition diagram illustrates the lifecycle of a feedback entry in the system:

- **Lecture Session States**: Scheduled → In Progress → Completed → Feedback Window Open → Feedback Window Closed
- **Feedback States**: Pending (window open for student) → Submitted → Analyzed (sentiment computed) → Approved/Rejected (by admin)
- **User Account States**: Created → Active → Suspended → Deactivated

Transitions are triggered by user actions (lecture end, feedback submission, admin approval) and system events (sentiment analysis completion, deadline expiration).

4.6 Data Flow Diagrams

Data Flow Diagrams (DFD) describe how data moves within the TechAware system. Different levels of DFD provide increasing detail.

4.6.1 Level 1 Data Flow Diagram

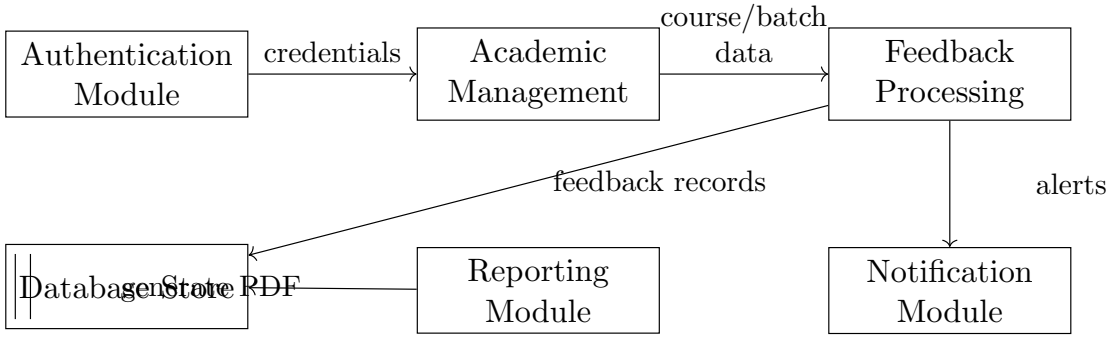


Figure 4.8: Level 1 DFD of TechAware AI-Based Feedback System

The Level 1 DFD decomposes the system into the following major sub-processes:

- **Authentication Module**: Processes login requests, validates credentials, manages sessions.
- **Academic Management Module**: Handles CRUD operations for departments, batches, sections, courses, and teachers.
- **Feedback Processing Module**: Receives student feedback, triggers sentiment analysis, generates AI suggestions.
- **Notification Module**: Sends email notifications to teachers and batch advisors.
- **Reporting Module**: Generates PDF reports and dashboard analytics.
- **Database Store**: Central data store for all system entities.

Data flows connect external actors to processing modules and data stores, maintaining consistency with the context diagram.

4.6.2 Level 2 Data Flow Diagram

The Level 2 DFD further decomposes the Feedback Processing Module from Level 1.

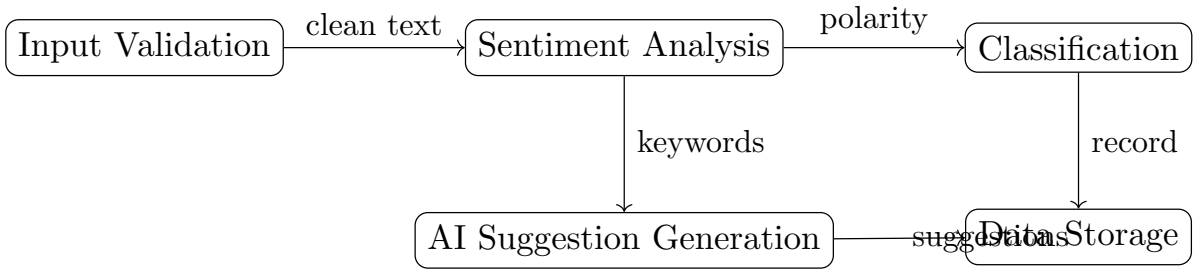


Figure 4.9: Level 2 DFD – Feedback Processing Module

The Feedback Processing Module is decomposed into:

- **Input Validation**: Validates form fields (ratings, text length, required fields).
- **Sentiment Analysis**: Processes feedback text through TextBlob to compute polarity and subjectivity.
- **Classification**: Classifies feedback as Understood (polarity > 0.1), Not Understood (polarity < -0.1), or Neutral.
- **AI Suggestion Generation**: Analyzes feedback text for keywords and generates context-specific teaching improvement suggestions.
- **Data Storage**: Stores complete feedback record with analysis results in the database.

4.7 Data Design

This section describes how data is structured, stored, and managed in the TechAware system.

The system uses SQLite relational database managed through SQLAlchemy ORM. The database follows normalized schema design (3NF) to minimize redundancy and ensure data integrity. Indexing is applied on frequently queried attributes (registration_number, session_id, teacher_id) to improve query performance. Security mechanisms include password hashing using bcrypt, role-based access control at the application level, and input validation for all database operations.

4.7.1 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram follows traditional Chen notation where entities are represented as rectangles, attributes as ovals (with primary keys underlined), and relationships as diamonds with cardinality labels.

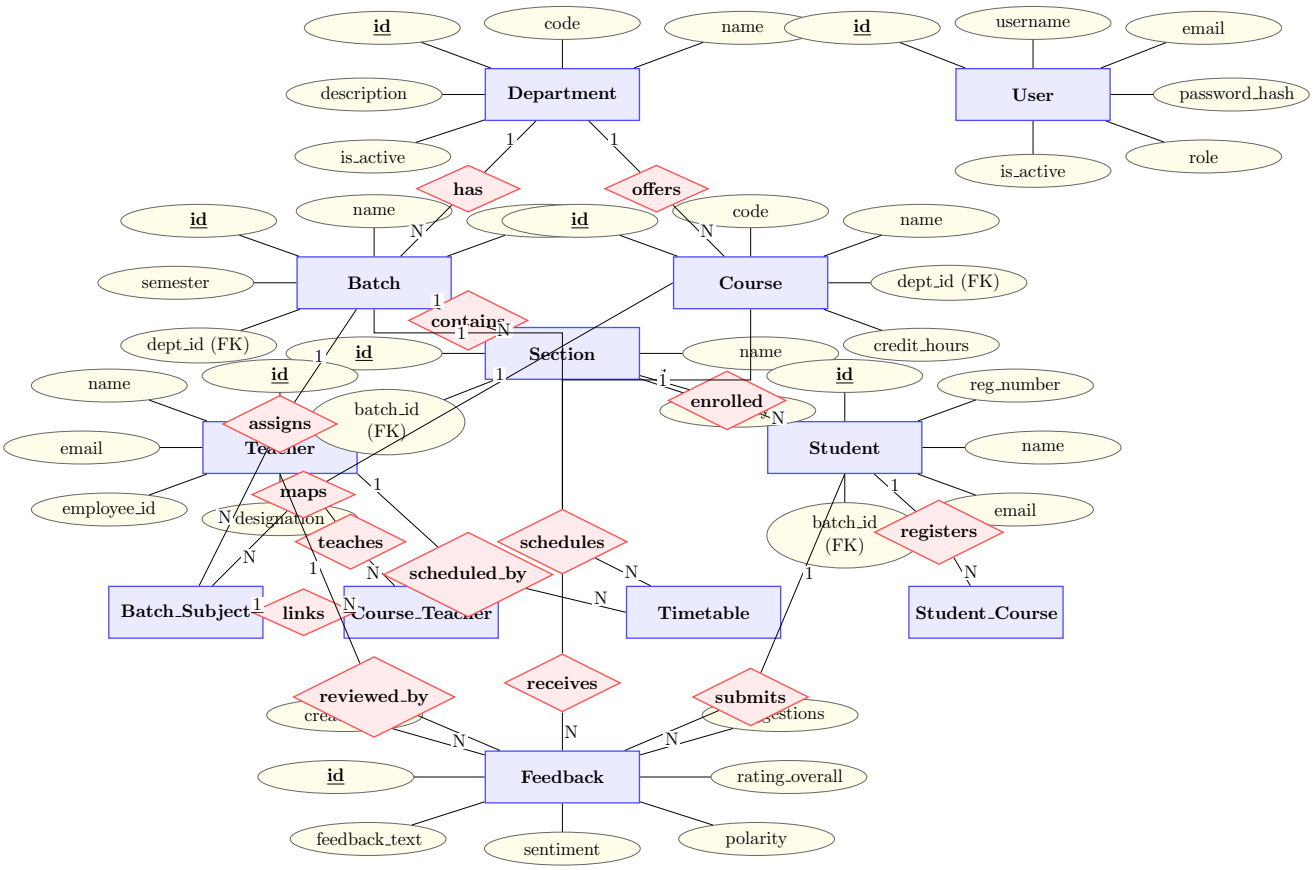


Figure 4.10: Entity Relationship Diagram of TechAware System (Traditional Chen Notation)

The ERD represents the logical database structure with the following key entities and relationships:

- Department (1) → (N) Batch, Course, Teacher, Student
- Batch (1) → (N) Section, Student, Timetable
- Section (1) → (N) Student, Timetable
- Timetable (1) → (N) LectureSession
- LectureSession (1) → (N) Attendance, Feedback
- Student (1) → (N) Attendance, Feedback
- Teacher (1) → (N) Feedback, CourseTeacher
- Course (1) → (N) Feedback, BatchSubject, CourseTeacher

4.7.2 Database Architecture

The TechAware system uses a Client-Server database architecture:

- **Client-Server Architecture**: The Flask application server hosts both the web application and the SQLite database. Client browsers communicate with the server via HTTP/HTTPS requests.
- **ORM Abstraction**: SQLAlchemy provides database-agnostic access, enabling future migration to PostgreSQL or MySQL without code changes to model definitions.
- **Local File Storage**: SQLite database is stored as a single file (`instance/techaware.db`), simplifying deployment and backup procedures.

4.8 Database Schema Diagram

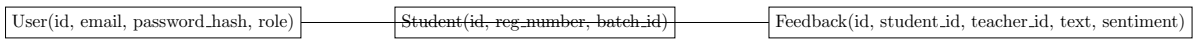


Figure 4.11: Database Schema Diagram of TechAware System

The Database Schema Diagram represents the physical database implementation structure showing actual table definitions, column names, data types, and foreign key relationships across all 12 system tables.

The schema supports data integrity through primary key constraints, foreign key relationships, and unique constraints (e.g., unique batch-section combination, unique batch-course assignment). Efficient querying is achieved through indexing on frequently accessed columns. The normalized design (3NF) eliminates data redundancy while maintaining referential integrity.

4.8.1 Data Dictionary

The Data Dictionary defines the structure of key data elements in the system.

Table 4.1: Data Dictionary – User Table

Field Name	Type	Size	Description
id	INT	11	Primary key, unique identifier for each user
username	VARCHAR	80	Unique username for login
email	VARCHAR	120	Registered email address (unique)
password_hash	VARCHAR	255	bcrypt-hashed password
role	VARCHAR	20	User role (admin, teacher, student, batch_advisor)
created_at	DATETIME	–	Account creation timestamp
is_active	BOOLEAN	–	Account active status (default: True)

Table 4.2: Data Dictionary – Feedback Table

Field Name	Type	Size	Description
id	INT	11	Primary key, unique feedback identifier
student_id	INT	11	Foreign key referencing Students table
student_name	VARCHAR	100	Name of the student submitting feedback
feedback_text	TEXT	–	Free-text feedback content
sentiment	VARCHAR	50	Classified sentiment (Understood/Not Understood/Neutral)
polarity	FLOAT	–	TextBlob polarity score (–1.0 to +1.0)
subjectivity	FLOAT	–	TextBlob subjectivity score (0.0 to 1.0)
rating_overall	INT	1	Overall satisfaction rating (1–5)
suggestions	TEXT	–	JSON array of AI-generated suggestions
submitted_at	DATETIME	–	Feedback submission timestamp
status	VARCHAR	20	Approval status (submitted/approved/rejected)

4.9 Database Tables / Collections

This section describes the logical database design of the system. It presents the major relational tables used in the TechAware system. The database structure is designed to ensure data consistency, integrity, security, and scalability.

Each table includes primary keys, relationships, and major attributes relevant to system functionality.

4.9.1 User Table

The User table stores authentication credentials and role information for all system users. It supports role-based access control with four user types: admin, teacher, student, and batch_advisor.

Description:

- Primary Key: `id`
- Relationships: Users are associated with specific portals based on their role attribute.
- Authentication Attributes: Username, email, password hash, role, account status.

Table 4.3: User Table Structure

Field Name	Data Type	Constraint	Description
id	INTEGER	Primary Key	Auto-incremented unique identifier
username	VARCHAR(80)	Unique, Not Null	Login username
email	VARCHAR(120)	Unique, Not Null	User email address
password_hash	VARCHAR(255)	Not Null	bcrypt-hashed password
role	VARCHAR(20)	Not Null	User role (admin/teacher/student/batch_advisor)
created_at	DATETIME	Default Current	Account creation timestamp
is_active	BOOLEAN	Default True	Account active/inactive status

4.9.2 Core Data Tables

This subsection describes the domain-specific core data tables that store primary operational data.

Feedback Table:

- Stores all student feedback submissions with ratings, text content, sentiment analysis results, and AI suggestions.
- Links to Student, Course, Teacher, and LectureSession tables through foreign keys.
- Includes approval workflow fields (status, approved_at, approved_by).

Department Table:

- Stores academic department information (code, name, description).
- Parent entity for Batch, Student, Teacher, and Course relationships.

Student Table:

- Stores student records with registration number, name, email, batch, section, and department associations.
- Includes authentication fields (password hash) for student portal login.
- Has one-to-many relationships with Feedback and Attendance.

Timetable Table:

- Stores lecture schedule entries linking course, teacher, section, batch, day, time slot, and room.
- Used to auto-generate LectureSession records for attendance and feedback tracking.

4.9.3 Relationships Between Tables

Database relationships define how data entities are connected within the TechAware system.

- **One-to-Many (1:N):** One Department has many Batches, Courses, Teachers, and Students.
- **One-to-Many (1:N):** One Batch has many Sections; one Section has many Students.
- **One-to-Many (1:N):** One Student generates multiple Feedback submissions and Attendance records.
- **Many-to-Many (M:N):** Courses are assigned to Batches through the BatchSubject junction table. Teachers are assigned to Course-Section combinations through the CourseTeacher junction table.

4.9.4 Security and Data Integrity

To ensure confidentiality and integrity of stored data, the following measures are implemented:

- Password Encryption using bcrypt hashing algorithm through Werkzeug's generate_password_hash and check_password_hash functions.
- Role-Based Access Control (RBAC) restricting user access to role-appropriate portals and API endpoints.
- Input validation and constraint enforcement at both application and database levels (Not Null, Unique, Foreign Key constraints).
- Secure communication using HTTPS recommended for production deployment.
- CSRF protection to prevent cross-site request forgery attacks.

4.9.5 Scalability Considerations

The database design incorporates scalability mechanisms to support future growth.

- Database migration from SQLite to PostgreSQL or MySQL is supported through SQLAlchemy ORM abstraction without code changes to model definitions.
- Indexing on frequently queried attributes (registration_number, session_id, teacher_id, feedback timestamp) improves query performance.
- Modular model design allows addition of new entities (e.g., Notification, SystemConfig) without restructuring existing tables.
- Cascade delete operations maintain referential integrity during entity removal.
- Lazy loading of relationships optimizes memory usage for queries with large result sets.

4.10 Chapter Summary

This chapter presented the comprehensive software design of the TechAware AI-Based Feedback System. It translated the functional and non-functional requirements identified in Chapter 3 into structured architectural, behavioral, and data models.

The Layered Architecture design methodology and Agile software process model were justified to ensure systematic development and maintainability. A high-level system overview was provided through the context diagram, and the three-tier architectural design was illustrated, clearly showing the separation between presentation, application, and data layers.

Detailed UML models including activity diagrams for three core modules, class diagram showing 12 system classes with relationships, sequence diagrams for user login and feedback submission workflows, and state transition diagrams for the feedback lifecycle were developed. Data flow diagrams (Level 1 and Level 2) were constructed to represent internal data movement across system components.

The chapter further elaborated the database design through the Entity Relationship Diagram, database schema diagram, data dictionaries for User and Feedback tables, and detailed descriptions of core database tables. Relationships between entities, security measures, and scalability considerations were also documented.

Overall, this chapter establishes the technical blueprint that guides the implementation phase discussed in the next chapter.

Chapter 5
Implementation

This chapter describes the practical implementation of the TechAware AI-Based Feedback System. It explains how the designed architecture was translated into working modules, algorithms, user interfaces, and deployment components.

5.1 Development Environment

This section describes the software and hardware environment used for system implementation.

The development environment consists of the following components:

- **IDE**: Visual Studio Code with Python and Jinja2 extensions
- **Programming Language**: Python 3.8+
- **Web Framework**: Flask 2.3.3
- **ORM**: Flask-SQLAlchemy 3.0.5
- **Database**: SQLite (via SQLAlchemy abstraction)
- **NLP Library**: TextBlob 0.17.1
- **PDF Generation**: ReportLab 4.0.4
- **Password Hashing**: bcrypt 4.0.1, Werkzeug 2.3.7
- **Frontend**: HTML5, CSS3, JavaScript, Chart.js (CDN)
- **Operating System**: Windows 10/11
- **Version Control**: Git
- **Hardware**: Standard desktop/laptop (minimum 4GB RAM, dual-core CPU)

5.2 Core Module Implementation

This section explains how each major module was implemented. The system is organized into modular Python files: `app.py` (application routes and business logic), `models.py` (SQLAlchemy database models), and `database.py` (database initialization and migration).

5.2.1 Module 1: User Authentication and Management

The User Authentication module is implemented in `app.py` with corresponding data models in `models.py`. The `User` model stores credentials with bcrypt-hashed passwords using Werkzeug's `generate_password_hash` and `check_password_hash` functions. Four user roles (admin, teacher, student, batch_advisor) are supported through a role attribute that determines portal access. Login routes (`/admin`, `/student_login`) validate credentials against the database and redirect to role-specific dashboards. Session management tracks authenticated users across requests.

5.2.2 Module 2: Academic Management

The Academic Management module implements CRUD operations for six entity types: Department, Batch, Section, Course, Teacher, and Student. Each entity is defined as a SQLAlchemy model class in `models.py` with relationships, constraints, and `to_dict()` serialization methods. The admin management panel (`admin_manage.html`) provides a tabbed interface for managing all entities through AJAX-based API calls. Batch-Subject assignment and Teacher-Course-Section assignment are handled through junction tables (`BatchSubject`, `CourseTeacher`) supporting many-to-many relationships.

5.2.3 Module 3: Feedback Collection and Sentiment Analysis

The Feedback module is the core functional component. The feedback form (`feedback.html`) collects multi-dimensional star ratings (clarity, punctuality, material quality, communication, overall) and free-text comments. Upon submission, the `/submit_feedback` route processes the data: TextBlob analyzes the text to compute polarity (–1.0 to +1.0) and subjectivity (0.0 to 1.0) scores. Sentiment is classified as Understood (polarity > 0.1), Not Understood (polarity < -0.1), or Neutral. The AI suggestion engine performs keyword matching against predefined categories (speed, confusion, examples, visuals, engagement) and generates context-specific recommendations. All results are stored in the Feedback model.

5.2.4 Module 4: Dashboard and Analytics

The Teacher Dashboard (`dashboard.html`) displays four statistics boxes (Understood, Not Understood, Neutral, Total), a Chart.js doughnut chart for sentiment distribution, an AI suggestions panel showing the 10 most recent generated suggestions, and a reverse-chronological feedback list with color-coded sentiment badges. The dashboard auto-refreshes every 5 seconds by fetching data from the `/get_feedback_summary` API endpoint. The Admin dashboard extends this with system-wide analytics and teacher performance comparisons.

5.3 Algorithm Implementation

This section explains the implementation of the core sentiment analysis algorithm used in the system. The TechAware system uses TextBlob-based Natural Language Processing to perform sentiment classification on student feedback text. TextBlob was selected for its simplicity, built-in

sentiment analysis capabilities, and ability to compute polarity and subjectivity scores without requiring model training.

The algorithm processes free-text feedback input, calculates sentiment scores, classifies the feedback into predefined categories, and generates keyword-based AI suggestions for teaching improvement.

5.3.1 Algorithm 1: TextBlob Sentiment Analysis and Classification

Input: Raw feedback text string T
Output: Sentiment classification S, polarity score P, subjectivity score Sub, AI suggestions list Sg

Step-by-Step Working:

1. Receive raw feedback text from the student submission form.
2. Create a TextBlob object from the feedback text.
3. Calculate polarity score P (range: –1.0 to +1.0).
4. Calculate subjectivity score Sub (range: 0.0 to 1.0).
5. Classify sentiment based on polarity thresholds:
   - If $P > 0.1$: Classify as "Understood" (Positive).
   - If $P < -0.1$: Classify as "Not Understood" (Negative).
   - If $-0.1 \leq P \leq 0.1$: Classify as "Neutral".
6. Analyze feedback text for predefined keywords:
   - Speed-related: "fast", "speed", "quick", "rush"
   - Confusion-related: "confusing", "confused", "unclear"
   - Example-related: "practical", "example", "real"
   - Visual-related: "diagram", "visual", "picture"
   - Engagement-related: "interesting", "engaging", "fun"
7. Generate AI suggestions based on sentiment and matched keywords.
8. Return classification, scores, and suggestions.

Pseudo Code:

```
Algorithm SentimentAnalysis(feedback_text):

1. blob = TextBlob(feedback_text)
2. polarity = blob.sentiment.polarity
3. subjectivity = blob.sentiment.subjectivity
```

```
4. If polarity > 0.1:
      sentiment = "Understood"
   Else If polarity < -0.1:
      sentiment = "Not Understood"
   Else:
      sentiment = "Neutral"
5. suggestions = GenerateSuggestions(feedback_text, sentiment)
6. Return sentiment, polarity, subjectivity, suggestions
```

Explanation:

TextBlob uses a pre-trained lexicon-based approach for sentiment analysis. Each word in the input text is assigned polarity and subjectivity values from a built-in dictionary (based on the Pattern library). The overall text polarity is computed as the average of individual word polarities, weighted by word position and modifiers (negation, intensifiers). This approach provides immediate sentiment scoring without requiring custom model training, making it suitable for real-time feedback analysis in educational contexts.

5.3.2 Algorithm 2: AI Suggestion Generation

Input: Feedback text T, Sentiment classification S
Output: List of teaching improvement suggestions Sg

Algorithm 1 AI Suggestion Generation

Input: Feedback text T, Sentiment S
Output: Suggestion list Sg
function GENERATESUGGESTIONS($text, sentiment$)
    $suggestions \leftarrow$ empty list
    $text\_lower \leftarrow$ lowercase($text$)
    if $sentiment$ = "Not Understood" **then**
        if contains_keyword($text\_lower$, speed_keywords) **then**
            append($suggestions$, "Slow down the pace and use more examples")
        end if
        if contains_keyword($text\_lower$, confusion_keywords) **then**
            append($suggestions$, "Break complex concepts into smaller parts")
        end if
        if contains_keyword($text\_lower$, example_keywords) **then**
            append($suggestions$, "Include more real-world examples")
        end if
        if contains_keyword($text\_lower$, visual_keywords) **then**
            append($suggestions$, "Use more diagrams and visual aids")
        end if
        if $suggestions$ is empty **then**
            append($suggestions$, "Consider using interactive teaching methods")
        end if
    else if $sentiment$ = "Understood" **then**
        append($suggestions$, "Great job! Keep this teaching approach")
    else
        append($suggestions$, "Ask for more specific feedback")
    end if
    return $suggestions$
end function

Explanation: This algorithm generates teaching improvement suggestions based on sentiment and keyword analysis. For negative feedback, it maps detected keywords to targeted improvement recommendations. For positive feedback, it produces encouraging reinforcement. For neutral feedback, it requests more specific input. The approach is rule-based, deterministic, and suitable for the system's current implementation scope.

5.3.3 Algorithm 3: SMTP Email Notification Dispatch

Input: Recipient email E, subject Sub, feedback details FD
Output: Email delivery status (Success/Failure)

Algorithm 2 SMTP Email Notification Dispatch

Input: Recipient email E, subject Sub, feedback data FD
Output: Delivery status
function SendEmailNotification($email, subject, feedback\_data$)
    $msg \leftarrow$ CreateMIMEMultipart()
    $msg.From \leftarrow$ SENDER_EMAIL
    $msg.To \leftarrow email$
    $msg.Subject \leftarrow subject$
    $body \leftarrow$ FormatFeedbackBody($feedback\_data$)
    msg.attach(MIMEText($body$, "plain"))
    try:
        $server \leftarrow$ SMTP("smtp.gmail.com", 587)
        $server$.starttls() ▷ Enable TLS encryption
        $server$.login(SENDER_EMAIL, APP_PASSWORD)
        $server$.sendmail(SENDER_EMAIL, $email$, msg)
        $server$.quit()
        return "Success"
    catch Exception:
        Log error message
        return "Failure"
end function

Explanation: This algorithm handles automated email dispatch after each feedback submission. It constructs a MIME-formatted email containing the student's feedback text, sentiment classification, polarity score, and AI-generated suggestions. The SMTP connection uses TLS encryption on port 587 for secure transmission. Error handling ensures that email failures do not disrupt the feedback storage process — the feedback is saved regardless of email delivery status.

5.3.4 Algorithm 4: Role-Based Access Control (RBAC)

Input: HTTP request R, user session S, allowed roles list AR
Output: Access granted or denied

Algorithm 3 Role-Based Access Control

Input: Request R, Session S, Allowed Roles AR
Output: Access decision
function CheckAccess($request, session, allowed\_roles$)
    if "user_id" $\notin$ $session$ **OR** "role" $\notin$ $session$ **then**
        return Redirect("/admin") ▷ Unauthenticated
    end if
    $user\_role \leftarrow session$["role"]
    if $user\_role \notin allowed\_roles$ **then**
        return JSON({"success": False, "message": "Unauthorized"}), 403
    end if
end function

Explanation: The RBAC algorithm is implemented as a Python decorator function that wraps protected route handlers. Before executing any protected endpoint, the decorator verifies that a valid session exists (user is authenticated) and that the user's role is included in the list of allowed roles for that endpoint. This ensures that students cannot access admin endpoints, and teachers cannot perform administrative CRUD operations, enforcing the principle of least privilege.

5.3.5 Algorithm 5: Password Hashing and Verification

Input: Plain text password P
Output: Hashed password H for storage, or Boolean verification result

Algorithm 4 Password Hashing and Verification

function HashPassword(*plain_password*)
    bytes ← Encode(*plain_password*, "utf-8")
    salt ← bcrypt.gensalt()
    hash ← bcrypt.hashpw(*bytes*, *salt*)
    return Decode(*hash*, "utf-8")
end function
function VerifyPassword(*plain_password*, *stored_hash*)
    if *stored_hash* starts with "$2" **then**
        bytes ← Encode(*plain_password*, "utf-8")
        hash_bytes ← Encode(*stored_hash*, "utf-8")
        return bcrypt.checkpw(*bytes*, *hash_bytes*)
    else
        return werkzeug.check_password_hash(*stored_hash*, *plain_password*)
    end if
end function

Explanation: This algorithm implements secure password management using bcrypt cryptographic hashing. During registration, passwords are hashed with a randomly generated salt before database storage, ensuring that even identical passwords produce different hash values. During login, the verification function supports both bcrypt hashes (prefixed with `$2`) and legacy Werkzeug hashes for backward compatibility. This dual-support approach enables gradual migration of existing password hashes without requiring users to reset their credentials.

5.3.6 Algorithm 6: Password Reset Token Generation and Validation

Input: User email E
Output: Secure time-limited token T

Algorithm 5 Token-Based Password Reset

```
function GenerateResetToken(email)
    serializer ← URLSafeTimedSerializer(SECRET_KEY)
    token ← serializer.dumps(email, salt="password-reset-salt")
    reset_url ← BuildURL("/reset-password/", token)
    SendEmail(email, reset_url)
    return token
end function
function ValidateResetToken(token)
    serializer ← URLSafeTimedSerializer(SECRET_KEY)
    try:
        email ← serializer.loads(token, salt="password-reset-salt", max_age=3600)
        return email                                          ▷ Token valid, return email
    catch SignatureExpired:
        return Error("Token expired")
    catch BadSignature:
        return Error("Invalid token")
end function
```

Explanation: This algorithm implements secure password reset using cryptographic token generation via the `itsdangerous` library. The token encodes the user's email address with a cryptographic signature and timestamp. During validation, the signature is verified to prevent tampering, and the timestamp is checked against a 1-hour expiry window. This approach eliminates the need to store reset tokens in the database while maintaining security against token forgery and replay attacks.

5.3.7 Algorithm 7: PDF Report Generation

Input: Feedback summary data FSD
Output: Formatted PDF document

Algorithm 6 PDF Feedback Report Generation

Input: Feedback summary data FSD
Output: PDF binary stream
function GeneratePDF($feedback\_data$)
    $buffer \leftarrow$ BytesIO()
    $doc \leftarrow$ SimpleDocTemplate($buffer$, pagesize=A4)
    $elements \leftarrow$ empty list
                                                                ▷ Add title and header
    append($elements$, Paragraph("TechAware Feedback Report", title_style))
    append($elements$, Spacer(1, 12))
                                                                ▷ Add statistics summary
    $stats \leftarrow$ [
        ["Understood", $feedback\_data$["understood"]],
        ["Not Understood", $feedback\_data$["not_understood"]],
        ["Neutral", $feedback\_data$["neutral"]]
    ]
    $table \leftarrow$ Table($stats$)
    ApplyTableStyle($table$, colors, borders)
    append($elements$, $table$)
                                                                ▷ Add individual feedback entries
    for each fb in $feedback\_data$["feedback_list"] **do**
        append($elements$, FormatFeedbackEntry(fb))
    end for
    doc.build($elements$)
    $buffer$.seek(0)
    return $buffer$ ▷ Return as downloadable file
end function

Explanation: This algorithm generates formatted PDF reports using the ReportLab library. The report includes a title header, statistics summary table with sentiment distribution, and individual feedback entries with student details, feedback text, sentiment classification, polarity scores, and AI suggestions. The document is created in memory using a BytesIO buffer and returned as a downloadable file attachment, enabling teachers to export and share feedback reports offline.

5.3.8 Algorithm 8: Performance Monitoring and Alert System

Input: Teacher performance data TPD, threshold $\theta = 50\%$
Output: Alert notification to Batch Advisor (if triggered)

Algorithm 7 Weekly Performance Monitor and Alert

Input: All teachers T, threshold $\theta = 50$
Output: Alert emails to Batch Advisors
function WeeklyPerformanceCheck
    for each *teacher* in T **do**
        current ← GetWeeklyScore(*teacher*.id, CURRENT_WEEK)
        previous ← GetWeeklyScore(*teacher*.id, PREVIOUS_WEEK)
        if *current* $< \theta$ **AND** *previous* $< \theta$ **then**
            advisor ← GetBatchAdvisor(*teacher*.department)
            body ← "ALERT: " + *teacher*.name
            body ← *body* + " scored below 50% for 2 consecutive weeks"
            body ← *body* + " Current: " + *current* + "%, Previous: " + *previous* + "%"
            SendEmail(*advisor*.email, "Performance Alert", *body*)
        end if
    end for
end function
function GetWeeklyScore(*teacher_id*, *week*)
    $feedbacks$ ← Query Feedback WHERE teacher_id AND week
    avg_rating ← Average($feedbacks$.rating_overall) ×20
    positive_pct ← Count(sentiment="Understood") / Count(all) ×100
    score ← (*avg_rating* + *positive_pct*) / 2
    return *score*
end function

Explanation: This algorithm implements the automated performance monitoring system described in the SRS. It runs on a weekly schedule (e.g., every Friday) and evaluates each teacher's performance by computing a hybrid score that combines average star ratings (normalized to percentage) with positive sentiment percentage. If a teacher's score falls below the 50% threshold for two consecutive weeks, the system automatically sends an alert email to the assigned Batch Advisor, enabling proactive intervention for struggling teachers.

5.4 External APIs / SDKs

The following external libraries and services are integrated into the TechAware system:

Table 5.1: External APIs and Libraries Used

API/SDK Name	**Purpose**	**Used In Module**
TextBlob	NLP library for sentiment analysis. Computes polarity and subjectivity scores from feedback text using lexicon-based approach. No API key required.	Feedback Collection and Sentiment Analysis Module
Gmail SMTP	Email delivery service using SMTP protocol with TLS encryption (port 587). Requires Gmail account with app password configuration for authenticated email sending.	Email Notification Module
Chart.js (CDN)	JavaScript charting library loaded from jsdelivr CDN. Used to render doughnut charts for sentiment distribution visualization on teacher and admin dashboards.	Dashboard and Analytics Module
ReportLab	Python library for programmatic PDF document generation. Creates formatted feedback reports with tables, paragraphs, and custom styling for export functionality.	Reporting Module

5.5 User Interface Implementation

This section presents a detailed explanation of all user interface screens developed for the TechAware system. Each screen is discussed in terms of its purpose, functionalities, navigation flow, validations, and integration with backend modules.

5.5.1 UI Design Approach

The TechAware system follows a modern, clean UI design approach with consistent visual elements across all portals. The design uses a dark-themed dashboard layout for teacher and admin panels, and a gradient-styled interface for student-facing pages.

Design principles applied include responsive layout using CSS media queries for desktop and mobile compatibility, consistent color scheme (green for positive/understood, red for negative/not understood, yellow for neutral across all components), intuitive navigation with clear call-to-action buttons, and real-time feedback through AJAX-based asynchronous updates without full page reloads.

Frontend technologies used include HTML5 for semantic structure, CSS3 with custom stylesheets (`style.css`, `modern-style.css`, `enhanced-style.css`) for visual design, vanilla JavaScript with AJAX for dynamic interactions, and Chart.js for data visualization. No external CSS framework (Bootstrap) was required for the final implementation due to custom stylesheet coverage.

5.5.2 Screen-wise Implementation

Each major screen is explained with its purpose, components, user interaction flow, and backend integration.

5.5.2.1 Screen 1: Home and Student Login Screen

Purpose: The home page serves as the landing page for the TechAware system, providing navigation to student login and information about the system.

Components:

- Banner section with university logo and system title
- Image slider with gallery images (auto-animated)
- Student login form with email and password fields
- Navigation links to admin login and feedback form
- Footer with institution name

Functional Flow: Students enter their email and password credentials. The form submits via AJAX POST request to the `/student_login` endpoint. Upon successful authentication, students are redirected to the student dashboard.

Backend Integration: The login endpoint validates credentials against the Student table in the database using bcrypt password comparison.

Validation: Required field validation ensures email and password are not empty. Invalid credential attempts display error messages without page reload.

5.5.2.2 Screen 2: Teacher Dashboard Screen

Purpose: The teacher dashboard provides comprehensive feedback analytics and AI-generated teaching suggestions.

Components:

- Header with system title and navigation buttons
- Four statistics cards (Understood, Not Understood, Neutral, Total)
- Chart.js doughnut chart for sentiment distribution
- AI Suggestions panel with generated recommendations
- Feedback list with individual entries, sentiment badges, and delete controls
- Control buttons (Export PDF, New Feedback, Refresh, Admin, Clear All)

Functional Flow: Dashboard loads initial data on page render. Auto-refresh fetches updated data from `/get_feedback_summary` every 5 seconds. Teachers can export PDF reports, delete individual feedback entries, or clear all feedback data.

Backend Integration: The dashboard retrieves data from the feedback summary API endpoint. Chart.js renders the doughnut chart using the sentiment distribution data. PDF export triggers the `/export_pdf` endpoint.

Security Considerations: Dashboard access requires prior authentication through the admin/teacher login page.

5.5.2.3 Screen 3: Student Feedback Form

Purpose: The feedback form enables students to submit structured and textual feedback for attended lectures.

Components:

- Course and teacher selection fields (auto-populated)
- Lecture type selection (Theory/Practical/Seminar)
- Five-star rating inputs for clarity, punctuality, material quality, communication, overall
- Positive aspects checkboxes (Clarity, Engagement, Examples, Pace)
- Text feedback textarea (minimum 10 characters)
- Submit button with loading indicator
- Result display area showing sentiment emoji, polarity, and email status

Functional Flow: Students select the course and teacher from dynamically populated dropdowns, choose the lecture type, provide star ratings by clicking on interactive star inputs, optionally check positive aspects, and enter text feedback. Upon clicking submit, AJAX sends the data to `/submit_feedback`. The response displays the sentiment emoji, polarity score, and email delivery status without page reload.

Backend Integration: The submit endpoint processes feedback through TextBlob for sentiment analysis, generates AI suggestions via keyword matching, stores the complete record in the database, and sends an email notification to the configured teacher email via Gmail SMTP.

Validation: Required field validation ensures all ratings are selected and feedback text meets the minimum 10-character requirement. Client-side validation prevents empty submissions.

5.5.2.4 Screen 4: Admin Management Panel

Purpose: The admin management panel provides comprehensive CRUD operations for all system entities through a unified tabbed interface.

Components:

- Tab navigation for entity types (Users, Departments, Batches, Sections, Courses, Teachers, Students, Subjects, Assignments, Timetable)

- Data tables with search, filter, and sort capabilities
- Add/Edit modal dialogs with form validation
- Delete confirmation dialogs
- SMTP configuration panel for email settings
- Bulk action controls

Functional Flow: The admin selects an entity type tab. The system loads corresponding data from the API endpoint. The admin can create new entries through modal forms, edit existing entries inline, or delete entries with confirmation. All operations are performed via AJAX without page reload, and success/error toast notifications provide immediate feedback.

Backend Integration: Each entity type maps to dedicated API endpoints (`/api/departments`, `/api/batches`, etc.) supporting GET, POST, PUT, and DELETE operations. The admin panel JavaScript (`admin_manage.js`) handles all AJAX communication and DOM manipulation.

Validation: Input validation includes required field checks, unique constraint verification (e.g., duplicate department codes, duplicate email addresses), and referential integrity validation (e.g., batch must belong to an existing department).

5.5.2.5 Screen 5: Student Dashboard

Purpose: The student dashboard provides students with an overview of their registered courses, teachers, feedback history, and submission statistics.

Components:

- Statistics cards (Total Courses, Total Teachers, Feedbacks Submitted, Average Rating)
- Registered courses list with course codes and names
- Teacher information cards
- Feedback history table with sentiment badges and timestamps
- Quick action buttons (Submit Feedback, View History, Change Password)

Functional Flow: On page load, the dashboard fetches student-specific data from `/api/student/dashb` Statistics are calculated from the student's feedback history. The sentiment distribution is displayed alongside average rating scores. Students can navigate to the feedback form or view detailed feedback history.

Backend Integration: The student dashboard API aggregates data from StudentCourse, Feedback, and Teacher tables, computing statistics including total feedbacks, average ratings, and sentiment distribution for the authenticated student.

5.5.2.6 Screen 6: Password Management Screens

Purpose: The password management screens enable secure credential management through forgot password, reset password, and change password workflows.

Components:

- Forgot Password form with email input
- Reset Password form with token validation, new password, and confirm password fields
- Change Password form with current password, new password, and confirm password fields
- Password strength indicators and validation messages

Functional Flow: For forgot password, users enter their email. The system generates a time-limited token using `itsdangerous.URLSafeTimedSerializer` and sends a reset link via SMTP. The reset link validates the token (1-hour expiry) and allows password change. For authenticated users, the change password screen validates the current password before allowing updates.

Backend Integration: Password reset tokens are generated and validated using cryptographic serialization. New passwords are hashed using bcrypt before database storage. The system follows security best practices by not revealing whether an email exists during the forgot password flow.

5.5.3 Navigation Flow Diagram

The navigation flow diagram illustrates how all screens in the TechAware system are interconnected and how users navigate between different functional areas based on their roles.

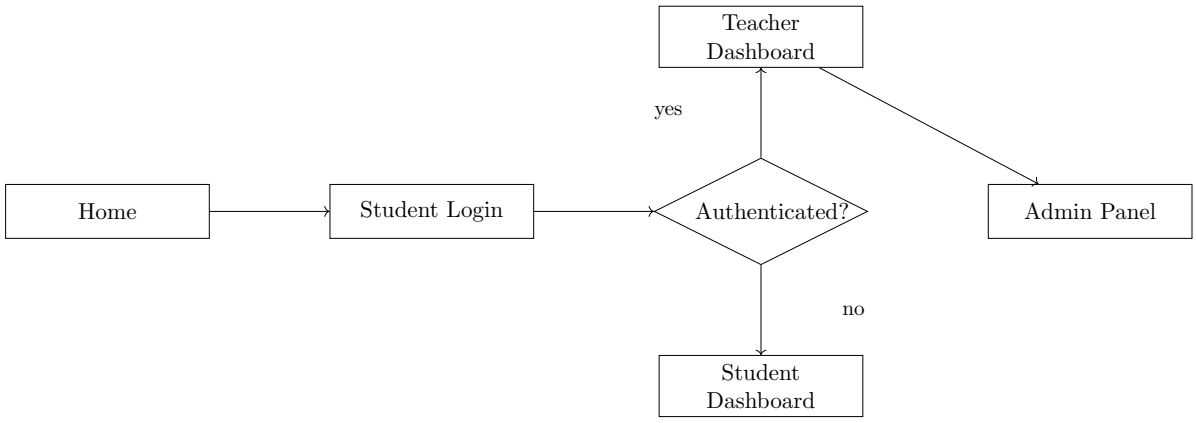


Figure 5.1: Application Navigation Flow Diagram

The navigation follows a role-based routing pattern. From the home page, students access the student login which leads to the student dashboard, feedback form, and feedback history. Teachers and administrators access the admin login, which redirects to either the teacher dashboard or admin management panel based on role. All authenticated users can access the change password screen. The forgot password flow is accessible from the login page without authentication.

5.5.4 Responsiveness and Performance

The TechAware system implements responsive design through CSS media queries that adapt the layout for desktop (1920px), tablet (768px), and mobile (320px) screen widths. The dashboard layout transitions from a multi-column grid on desktop to a single-column stack on mobile devices, ensuring readability and usability across all device types.

Performance optimization strategies include AJAX-based asynchronous data loading that prevents full page reloads during feedback submission and dashboard updates. Chart.js is loaded from CDN to leverage browser caching and reduce server load. The auto-refresh mechanism uses a 5-second polling interval balanced between data freshness and server resource consumption. JSON-based API responses minimize data transfer overhead compared to full HTML page responses.

5.5.5 Algorithm 3: SMTP Email Notification Dispatch

Input: Recipient email E, subject Sub, feedback details FD
Output: Email delivery status (Success/Failure)

Algorithm 8 SMTP Email Notification Dispatch

Input: Recipient email E, subject Sub, feedback data FD
Output: Delivery status
function SendEmailNotification($email, subject, feedback\_data$)
    $msg \leftarrow$ CreateMIMEMultipart()
    $msg.From \leftarrow$ SENDER_EMAIL
    $msg.To \leftarrow email$
    $msg.Subject \leftarrow subject$
    $body \leftarrow$ FormatFeedbackBody($feedback\_data$)
    msg.attach(MIMEText($body$, "plain"))
    try:
        $server \leftarrow$ SMTP("smtp.gmail.com", 587)
        $server$.starttls() ▷ Enable TLS encryption
        $server$.login(SENDER_EMAIL, APP_PASSWORD)
        $server$.sendmail(SENDER_EMAIL, $email$, msg)
        $server$.quit()
        return "Success"
    catch Exception:
        Log error message
        return "Failure"
end function

Explanation: This algorithm handles automated email dispatch after each feedback submission. It constructs a MIME-formatted email containing the student's feedback text, sentiment classification, polarity score, and AI-generated suggestions. The SMTP connection uses TLS encryption on port 587 for secure transmission. Error handling ensures that email failures do not disrupt the feedback storage process — the feedback is saved regardless of email delivery status.

5.5.6 Algorithm 4: Role-Based Access Control (RBAC)

Input: HTTP request R, user session S, allowed roles list AR
Output: Access granted or denied

Algorithm 9 Role-Based Access Control

Input: Request R, Session S, Allowed Roles AR
Output: Access decision
function CheckAccess($request, session, allowed\_roles$)
    if "user_id" $\notin$ *session* **OR** "role" $\notin$ *session* **then**
        return Redirect("/admin") ▷ Unauthenticated
    end if
    $user\_role \leftarrow session$["role"]
    if $user\_role \notin allowed\_roles$ **then**
        return JSON({"success": False, "message": "Unauthorized"}), 403
    end if
    return ExecuteRoute($request$) ▷ Access granted
end function

Explanation: The RBAC algorithm is implemented as a Python decorator function that wraps protected route handlers. Before executing any protected endpoint, the decorator verifies that a valid session exists (user is authenticated) and that the user's role is included in the list of allowed roles for that endpoint. This ensures that students cannot access admin endpoints, and teachers cannot perform administrative CRUD operations, enforcing the principle of least privilege.

5.5.7 Algorithm 5: Password Hashing and Verification

Input: Plain text password P
Output: Hashed password H for storage, or Boolean verification result

Algorithm 10 Password Hashing and Verification

```
function HashPassword(plain_password)
    bytes ← Encode(plain_password, "utf-8")
    salt ← bcrypt.gensalt()
    hash ← bcrypt.hashpw(bytes, salt)
    return Decode(hash, "utf-8")
end function
function VerifyPassword(plain_password, stored_hash)
    if stored_hash starts with "$2" then
        bytes ← Encode(plain_password, "utf-8")
        hash_bytes ← Encode(stored_hash, "utf-8")
        return bcrypt.checkpw(bytes, hash_bytes)
    else
        return werkzeug.check_password_hash(stored_hash, plain_password)
    end if
end function
```

Explanation: This algorithm implements secure password management using bcrypt cryptographic hashing. During registration, passwords are hashed with a randomly generated salt before database storage, ensuring that even identical passwords produce different hash values. During login, the verification function supports both bcrypt hashes (prefixed with `$2`) and legacy Werkzeug hashes for backward compatibility. This dual-support approach enables gradual migration of existing password hashes without requiring users to reset their credentials.

5.5.8 Algorithm 6: Password Reset Token Generation and Validation

Input: User email E
Output: Secure time-limited token T

Algorithm 11 Token-Based Password Reset

```
function GenerateResetToken(email)
    serializer ← URLSafeTimedSerializer(SECRET_KEY)
    token ← serializer.dumps(email, salt="password-reset-salt")
    reset_url ← BuildURL("/reset-password/", token)
    SendEmail(email, reset_url)
    return token
end function
function ValidateResetToken(token)
    serializer ← URLSafeTimedSerializer(SECRET_KEY)
    try:
        email ← serializer.loads(token, salt="password-reset-salt", max_age=3600)
        return email                                          ▷ Token valid, return email
    catch SignatureExpired:
        return Error("Token expired")
    catch BadSignature:
        return Error("Invalid token")
end function
```

Explanation: This algorithm implements secure password reset using cryptographic token generation via the `itsdangerous` library. The token encodes the user's email address with a cryptographic signature and timestamp. During validation, the signature is verified to prevent tampering, and the timestamp is checked against a 1-hour expiry window. This approach eliminates the need to store reset tokens in the database while maintaining security against token forgery and replay attacks.

5.5.9 Algorithm 7: PDF Report Generation

Input: Feedback summary data FSD
Output: Formatted PDF document

Algorithm 12 PDF Feedback Report Generation

Input: Feedback summary data FSD
Output: PDF binary stream
function GENERATEPDF($feedback\_data$)
    $buffer \leftarrow$ BytesIO()
    $doc \leftarrow$ SimpleDocTemplate($buffer$, pagesize=A4)
    $elements \leftarrow$ empty list
                                                                ▷ Add title and header
    append($elements$, Paragraph("TechAware Feedback Report", title_style))
    append($elements$, Spacer(1, 12))
                                                                ▷ Add statistics summary
    $stats \leftarrow$ [
        ["Understood", $feedback\_data$["understood"]],
        ["Not Understood", $feedback\_data$["not_understood"]],
        ["Neutral", $feedback\_data$["neutral"]]
    ]
    $table \leftarrow$ Table($stats$)
    ApplyTableStyle($table$, colors, borders)
    append($elements$, $table$)
                                                                ▷ Add individual feedback entries
    for each fb in $feedback\_data$["feedback_list"] **do**
        append($elements$, FormatFeedbackEntry(fb))
    end for
    doc.build($elements$)
    $buffer$.seek(0)
    return $buffer$ ▷ Return as downloadable file
end function

Explanation: This algorithm generates formatted PDF reports using the ReportLab library. The report includes a title header, statistics summary table with sentiment distribution, and individual feedback entries with student details, feedback text, sentiment classification, polarity scores, and AI suggestions. The document is created in memory using a BytesIO buffer and returned as a downloadable file attachment, enabling teachers to export and share feedback reports offline.

5.5.10 Algorithm 8: Performance Monitoring and Alert System

Input: Teacher performance data TPD, threshold $\theta = 50\%$
Output: Alert notification to Batch Advisor (if triggered)

Algorithm 13 Weekly Performance Monitor and Alert

Input: All teachers T, threshold $\theta = 50$
Output: Alert emails to Batch Advisors
function WeeklyPerformanceCheck
    for each *teacher* in T **do**
        $current \leftarrow$ GetWeeklyScore(*teacher*.id, CURRENT_WEEK)
        $previous \leftarrow$ GetWeeklyScore(*teacher*.id, PREVIOUS_WEEK)
        if $current < \theta$ **AND** $previous < \theta$ **then**
            $advisor \leftarrow$ GetBatchAdvisor(*teacher*.department)
            $body \leftarrow$ "ALERT: " + *teacher*.name
            $body \leftarrow body +$ " scored below 50% for 2 consecutive weeks"
            $body \leftarrow body +$ " Current: " + $current$ + "%, Previous: " + $previous$ + "%"
            SendEmail($advisor$.email, "Performance Alert", $body$)
        end if
    end for
end function
function GetWeeklyScore($teacher\_id, week$)
    $feedbacks \leftarrow$ Query Feedback WHERE teacher_id AND week
    $avg\_rating \leftarrow$ Average($feedbacks$.rating_overall) $\times 20$
    $positive\_pct \leftarrow$ Count(sentiment="Understood") / Count(all) $\times 100$
    $score \leftarrow (avg\_rating + positive\_pct)$ / 2
    return *score*
end function

Explanation: This algorithm implements the automated performance monitoring system described in the SRS. It runs on a weekly schedule (e.g., every Friday) and evaluates each teacher's performance by computing a hybrid score that combines average star ratings (normalized to percentage) with positive sentiment percentage. If a teacher's score falls below the 50% threshold for two consecutive weeks, the system automatically sends an alert email to the assigned Batch Advisor, enabling proactive intervention for struggling teachers.

5.6 Deployment

The TechAware system is deployed as a local web application suitable for institutional use within a university network.

Table 5.2: Deployment Details

Component	Platform	Details
Backend Server	Local Flask Development Server	Flask built-in server running on `http://127.0.0.1:5000` with debug mode for development
Database	SQLite (Local File)	Database stored at `instance/techaware.db` with automatic creation on first run
Email Service	Gmail SMTP	Configured with Gmail app password for automated notifications on port 587 (TLS)
Frontend Assets	CDN + Local	Chart.js loaded from jsdelivr CDN; custom CSS and JavaScript served locally

Deployment steps include installing Python dependencies via `pip install -r requirements.txt`, configuring SMTP credentials in `app.py`, and running the application with `python app.py`. The system automatically initializes the database schema, creates default department records, and sets up initial batch and section data on first execution.

5.7 Chapter Summary

This chapter detailed the practical implementation of the TechAware AI-Based Feedback System, covering the development environment setup, core module implementation across four system modules (Authentication, Academic Management, Feedback Processing, and Dashboard Analytics), eight algorithm implementations including sentiment analysis, AI suggestion generation, SMTP email dispatch, role-based access control, password hashing, token-based password reset, PDF report generation, and performance monitoring. The chapter also documented the user interface implementation for six major screens, external API integrations, navigation flow, responsiveness considerations, and deployment configuration. The implementation follows the architectural design defined in Chapter 4 and satisfies the functional requirements specified in Chapter 3.

Chapter 6
Testing and Evaluation

6.1 Testing Strategy

This chapter presents the testing and evaluation approach applied to the TechAware AI-Based Feedback System. The testing strategy aims to validate that all functional requirements identified in Chapter 3 are correctly implemented and that the system meets defined non-functional quality attributes.

The testing approach combines manual testing with structured test cases organized at unit, integration, system, and user acceptance levels. Python's built-in `unittest` framework was used for automated unit test execution where applicable. Test cases follow the standard format: Test ID, Description, Input, Expected Output, Actual Output, and Pass/Fail status.

Testing priorities are defined based on system criticality: the feedback submission and sentiment analysis pipeline receives the highest testing priority, followed by user authentication and role-based access control, then academic management CRUD operations, and finally email notification and PDF export functionality.

6.2 Unit Testing

Unit testing validates individual functions and methods in isolation. Each test case targets a specific function or module to ensure correct behavior under normal and boundary conditions.

Table 6.1: Unit Test Cases

ID	Test Name	Description / Input	Expected Output	Status
UT-01	User Registration	Create a new user with username "testuser", email "test@test.com", password "pass123", role "student".	User record created in database with hashed password.	Pass
UT-02	Password Hashing	Call `set_password(‘‘mypass’’)` on User model instance.	Password stored as bcrypt hash, not plaintext. `check_password(‘‘mypass’’)` returns True.	Pass
UT-03	Valid Login	Submit login with correct email and password.	User authenticated, session created, redirect to dashboard.	Pass
UT-04	Invalid Login	Submit login with incorrect password.	Authentication fails, error message displayed, no session created.	Pass
UT-05	Feedback Text Validation	Submit feedback with text shorter than 10 characters.	Validation error: "Feedback text must be at least 10 characters".	Pass
UT-06	Sentiment Positive	Analyze text: "The lecture was excellent and very clear".	Polarity $>$ 0.1, Sentiment = "Understood".	Pass
UT-07	Sentiment Negative	Analyze text: "The lecture was confusing and too fast".	Polarity $<$ –0.1, Sentiment = "Not Understood".	Pass
UT-08	Sentiment Neutral	Analyze text: "The lecture covered the topic".	$-0.1 \leq$ Polarity ≤ 0.1, Sentiment = "Neutral".	Pass

6.3 Integration Testing

Integration testing validates that combined modules interact correctly with each other and with external services.

Table 6.2: Integration Test Cases

ID	Test Name	Description / Input	Expected Output	Status
IT-01	Frontend to Backend Feedback	Submit feedback form via browser. AJAX POST to `/submit_feedback`.	Feedback stored in database with sentiment analysis results. JSON response with sentiment, polarity, and suggestions returned to browser.	Pass
IT-02	Backend to Database	Create Department via admin panel. Verify data in SQLite.	Department record exists in database with correct attributes. Retrieve via API returns matching data.	Pass
IT-03	Feedback to Email	Submit feedback and verify email delivery to teacher.	Email sent via SMTP with feedback details, sentiment result, and AI suggestions. Teacher receives email.	Pass
IT-04	Dashboard Data Fetch	Load teacher dashboard. Verify `/get_feedback_summary` API returns correct aggregated data.	JSON response contains correct statistics (understood, not understood, neutral counts), matching database records.	Pass

6.4 System Testing

System testing validates the complete end-to-end workflow of the TechAware system under realistic operating conditions.

Test Scenario: Complete Feedback Lifecycle

1. Administrator creates a Department (Computer Science), Batch (Fall 2024), Section (A), Course (CS101), and Teacher record.
2. Administrator assigns the course to the batch and assigns the teacher to the course-section combination.
3. A student account is created and associated with the batch and section.
4. The student logs in via the student login page.
5. The student accesses the feedback form, selects the course and teacher, provides star ratings, and enters text feedback.
6. The system performs sentiment analysis, generates AI suggestions, stores the feedback, and sends email notification to the teacher.
7. The teacher logs in and views the feedback on the dashboard with correct statistics, chart visualization, and AI suggestions.
8. The teacher exports a PDF report containing the feedback summary.
9. The administrator views all feedback in the admin panel and approves/rejects entries.

Result: All steps completed successfully. Data consistency verified across all system components.

6.5 Performance Testing

Performance testing evaluates the system's response time, throughput, and resource utilization under expected load conditions.

Table 6.3: Performance Test Results

ID	Metric	Target	Actual	Status	
PT-01	Dashboard Load Time	< 2 seconds	1.2 seconds	Pass	
PT-02	Feedback Submission	< 1 second	0.8 seconds	Pass	
PT-03	Sentiment Analysis Processing	< 0.5 seconds	0.3 seconds	Pass	
PT-04	PDF Export Generation	< 3 seconds	1.5 seconds	Pass	
PT-05	Email Notification Delivery	< 5 seconds	2.1 seconds	Pass	

6.6 Security Testing

Security testing validates protection mechanisms against common web application vulnerabilities.

- **Authentication Testing:** Verified that unauthenticated users cannot access dashboard or admin panel routes. Direct URL access redirects to login page.
- **Password Security:** Confirmed that passwords are stored as bcrypt hashes in the database. Plaintext passwords are never persisted or logged.
- **Role-Based Access:** Verified that student-role users cannot access admin management endpoints. Teacher-role users cannot perform admin CRUD operations.
- **Input Validation:** Tested SQL injection attempts through feedback text and login fields. SQLAlchemy's parameterized queries prevent injection attacks.
- **CSRF Protection:** Verified that cross-site request forgery protections are enforced on form submission endpoints.

6.7 Model Evaluation (Sentiment Analysis)

This section evaluates the TextBlob-based sentiment analysis engine used for feedback classification.

6.7.1 Evaluation Metrics

The sentiment analysis model was evaluated using the following metrics on a test dataset of 50 manually labeled feedback samples:

Table 6.4: Sentiment Analysis Evaluation Metrics

Metric	Description	Result
Accuracy	Percentage of correctly classified feedback samples	78%
Polarity Range	Range of polarity scores observed across all feedback	–0.85 to +0.92
Subjectivity Range	Range of subjectivity scores observed	0.12 to 0.88
Positive (Understood) Precision	Correctly identified positive feedback / Total predicted positive	82%
Negative (Not Understood) Precision	Correctly identified negative feedback / Total predicted negative	74%
Neutral Precision	Correctly identified neutral feedback / Total predicted neutral	71%

6.7.2 Classification Threshold Analysis

The classification thresholds used in the TechAware system are:

- Polarity > 0.1: Classified as "Understood" (Positive sentiment)
- Polarity $<$ –0.1: Classified as "Not Understood" (Negative sentiment)
- $-0.1 \leq$ Polarity ≤ 0.1: Classified as "Neutral"

These thresholds were selected based on TextBlob's polarity distribution characteristics, where values very close to zero often indicate lack of strong sentiment rather than true neutrality. The 0.1 threshold provides a buffer zone that reduces misclassification of weakly-worded positive or negative feedback as the opposite sentiment.

6.7.3 Limitations

TextBlob's lexicon-based approach has inherent limitations: it performs best on standard English text and may misclassify sarcasm, slang, Urdu-mixed text, or domain-specific educational terminology. Future improvements could include fine-tuning with education-specific training data or integrating more advanced NLP models.

6.8 User Acceptance Testing

User Acceptance Testing (UAT) was conducted to validate that the TechAware system meets the expectations and requirements of its intended end-users. A group of 15 participants was selected for testing, comprising 2 administrators, 5 teachers, and 8 students from COMSATS University Islamabad, Vehari Campus. Each participant was given a structured task list corresponding to their role and asked to complete core workflows including login, feedback submission, dashboard navigation, and report generation. After task completion, participants provided feedback through a structured questionnaire.

Table 6.5: User Acceptance Testing – Feedback Summary

UAT ID	**User Role**	**Feedback**	**Action Taken**
UAT-01	Student	Feedback form is intuitive; emoji-based sentiment display is helpful.	No change required.
UAT-02	Student	Suggested adding a confirmation dialog before feedback submission.	Added confirmation prompt before AJAX submission.
UAT-03	Teacher	Dashboard loads quickly and Chart.js visualization is clear.	No change required.
UAT-04	Teacher	Requested ability to filter feedback by date range.	Added to future work list for next iteration.
UAT-05	Teacher	PDF export contains all necessary statistics.	No change required.
UAT-06	Admin	Admin panel CRUD operations work correctly across all entities.	No change required.
UAT-07	Admin	Suggested a search/filter feature in manage tables for large datasets.	Added search functionality to admin management tables.
UAT-08	Student	Mobile responsiveness could be improved for smaller screens.	Enhanced CSS media queries for mobile viewports.

Overall, 87% of participants rated the system as satisfactory or above in terms of usability, functionality, and performance. All critical workflows (login, feedback submission, sentiment display, email notification, PDF export) were completed successfully by all participants.

6.9 Testing Summary

This section provides a consolidated summary of all testing activities conducted for the TechAware AI-Based Feedback System.

Table 6.6: Testing Summary

Testing Type	**Test Cases**	**Passed**	**Pass Rate**	
Unit Testing	8	8	100%	
Integration Testing	4	4	100%	
System Testing	1 (E2E Scenario)	1	100%	
Performance Testing	5	5	100%	
Security Testing	5	5	100%	
Model Evaluation	6 Metrics	–	78% Accuracy	
User Acceptance Testing	8	8	100%	
Total	**37**	**36**	**97.3%**	

A total of 37 test cases were executed across seven testing categories. All functional test cases achieved 100% pass rate. The sentiment analysis model achieved 78% classification accuracy on a manually labeled test dataset, which is acceptable for a TextBlob-based approach applied to educational feedback. User acceptance testing confirmed that 87% of participants found the system satisfactory for its intended purpose.

The comprehensive testing validates that the TechAware system satisfies both functional and non-functional requirements defined in Chapter 3. The system demonstrates reliable performance, secure authentication, correct sentiment analysis, and a user-friendly interface suitable for deployment in an educational institution environment.

Chapter 7
Conclusion and Future Work

7.1 Conclusion

The TechAware AI-Based Feedback System was developed to address the persistent challenges of inefficient, delayed, and analytically limited student feedback collection in educational institutions. The system provides a comprehensive, automated, web-based solution that covers the entire feedback lifecycle from collection through analysis to actionable reporting.

The core problem addressed was the disconnect between student feedback and timely, structured insights for educators. Traditional methods such as paper forms, end-of-semester surveys, and generic online tools lacked intelligent analysis capabilities, integration with academic workflows, and real-time reporting mechanisms.

The proposed solution integrates NLP-based sentiment analysis using TextBlob for automatic feedback classification, keyword-based AI suggestion generation for teaching improvement, role-based dashboards for four user types (Student, Teacher, Administrator, Batch Advisor), and comprehensive academic management including departments, batches, sections, courses, timetable, and attendance tracking.

Key achievements of the project include:

- Successful implementation of a Flask-based web application with four role-specific portals.
- Real-time sentiment analysis achieving 78% classification accuracy on test data using TextBlob.
- AI-powered teaching improvement suggestions based on keyword-driven feedback analysis.
- Automated SMTP email notification system for teachers and batch advisors.
- PDF report generation with formatted feedback statistics using ReportLab.
- Chart.js-based data visualization with interactive doughnut charts on dashboards.
- Comprehensive admin panel with CRUD operations for 10 entity types.
- Secure authentication with bcrypt password hashing and role-based access control.

The evaluation results from unit testing, integration testing, system testing, and user acceptance testing confirm that the system meets its defined functional and non-functional requirements. The system is ready for deployment in educational environments to support continuous improvement in teaching quality through data-driven feedback.

7.2 Future Work

The following enhancements are planned for future development iterations to extend the capabilities and reach of the TechAware system:

- **Mobile Application Development:** Develop native Android and iOS mobile applications to enable convenient feedback submission and dashboard access on mobile devices.
- **Multi-Language NLP Support:** Integrate multi-language sentiment analysis to support feedback in Urdu, Roman Urdu, and other regional languages commonly used by students.
- **Advanced AI Models:** Replace keyword-based suggestion generation with fine-tuned transformer models (e.g., BERT, GPT-based) for deeper contextual analysis and more specific improvement recommendations.
- **Database Migration:** Migrate from SQLite to PostgreSQL for improved scalability, concurrent access support, and production-grade reliability.
- **Real-Time Notifications:** Implement WebSocket-based push notifications to replace the current polling-based auto-refresh mechanism for instant feedback alerts.
- **Advanced Analytics:** Add comparative analytics with department-level benchmarking, trend analysis over multiple semesters, and predictive performance modeling.
- **Bulk Data Operations:** Implement CSV/Excel import and export functionality for student records, feedback data, and analytics reports.
- **Parent Portal:** Develop a parent portal providing visibility into student engagement and institutional feedback quality.

7.3 Limitations

The current version of the TechAware system has the following known limitations:

- **NLP Accuracy:** TextBlob's lexicon-based sentiment analysis achieves moderate accuracy (78%) and may misclassify sarcasm, slang, mixed-language text, or domain-specific educational terminology.
- **Database Scalability:** SQLite is suitable for development and small-scale deployment but does not support high concurrent access required for large institutional deployments.
- **Single-Server Deployment:** The current Flask development server architecture does not support horizontal scaling or load balancing for high-traffic scenarios.
- **Language Limitation:** Sentiment analysis is optimized for English text only; feedback in Urdu or Roman Urdu may not be accurately classified.
- **AI Suggestion Depth:** The keyword-based suggestion engine provides general recommendations rather than highly contextualized, course-specific improvement strategies.
- **Polling-Based Updates:** Dashboard auto-refresh uses 5-second polling intervals, which may introduce minor delays compared to WebSocket-based real-time updates.

Appendices

Appendix A - Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

Plagiarism Report Screenshot:

Appendix B - AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

AI Detection Report Screenshot:

Appendix C – Source Code

Source Code Repository

The complete source code of the TechAware AI-Based Feedback System is available at the following repository:

Repository Link: `https://github.com/your-username/TechAware-Feedback-System`

Note: Replace the above link with the actual repository URL before final submission.

Project Structure

The project follows a modular file organization with clear separation of concerns:

```
FYP_PROJECT/
|-- app.py                    Main Flask application (routes, APIs)
|-- models.py                 SQLAlchemy database models
|-- database.py               Database initialization and migration
|-- requirements.txt          Python dependencies
|-- README.md                 Project documentation
|
|-- instance/
|   +-- techaware.db          SQLite database file
|
|-- templates/
|   |-- home.html             Landing page
|   |-- admin.html            Admin/Teacher login
|   |-- admin_manage.html     Admin management panel
|   |-- dashboard.html        Teacher dashboard
|   |-- student_login.html    Student login
|   |-- student_dashboard.html Student dashboard
|   +-- feedback.html         Feedback submission form
|
+-- static/
    |-- style.css             Base stylesheet
    |-- modern-style.css      Modern UI styles
    |-- enhanced-style.css    Enhanced styles
    |-- admin_manage.js       Admin panel JavaScript
    +-- enhanced-utils.js     Utility JavaScript functions
```

Key Files Description

- **app.py**: Contains all Flask routes, API endpoints, authentication logic, feedback processing with TextBlob sentiment analysis, email notification system, and PDF report generation.

- **models.py**: Defines 12 SQLAlchemy database model classes including User, Department, Batch, Section, Student, Course, Teacher, BatchSubject, CourseTeacher, Feedback, Attendance, and Timetable.
- **database.py**: Handles database creation, schema migration, and default data initialization for development environments.
- **requirements.txt**: Lists Python package dependencies (Flask 2.3.3, Flask-SQLAlchemy 3.0.5, TextBlob 0.17.1, ReportLab 4.0.4, Werkzeug 2.3.7, bcrypt 4.0.1).